ŽILINSKÁ UNIVERZITA
V ŽILINE

Fakulta riadenia a informatiky

Využitie regulárnych výrazov a konečných automatov pri tvorbe lexikálneho analyzátora pre základný imperatívny jazyk

Bakalárska práca

Jakub Gubány

Študijný program: Informatika
Študijný odbor: Informatika
Školiace pracovisko: Žilinská univerzita v Žiline,
Vedúci bakalárskej práce: Mgr. Peter Novotný, PhD.
Žilina 2026

ŽILINSKÁ UNIVERZITA V ŽILINE, FAKULTA RIADENIA A INFORMATIKY.

ZADANIE TÉMY BAKALÁRSKEJ PRÁCE.

Študijný program: Informatika

Meno a priezvisko	**Osobné číslo**
Jakub Gubány	561774

Názov práce v slovenskom aj anglickom jazyku

Uplatnenie regulárnych výrazov a konečných automatov pri tvorbe lexikálneho analyzátora a jeho praktické využitie

Application of regular expressions and finite automata in the development of a lexical analyzer and its practical use

Zadanie úlohy, ciele, pokyny pre vypracovanie (Ak je málo miesta, použite opačnú stranu)

Cieľ bakalárskej práce:
Cieľom práce je navrhnúť a implementovať vlastný systém lexikálnej analýzy založený na princípoch konečných automatov. Práca sa bude venovať formálnej špecifikácii tokenov pomocou regulárnych výrazov, ich prevodu na deterministické konečné automaty (DFA) a následnému rozpoznávaniu vstupných textových reťazcov. V teoretickej časti budú predstavené základné koncepty konečných automatov a regulárnych jazykov, ktoré tvoria základ lexikálnej analýzy. Praktická časť sa zameria na vývoj funkčného lexikálneho analyzátora, ktorý bude porovnaný s existujúcimi nástrojmi, ako sú Flex alebo ANTLR, z hľadiska efektívnosti a použiteľnosti. Výsledkom bude modulárny a rozšíriteľný lexikálny analyzátor, ktorý môže slúžiť ako základ pre ďalší vývoj nástrojov na spracovanie programovacích jazykov alebo iných textových formátov.

Obsah:
- Kontext a význam témy, motivácia výberu témy.
- Teoretické východisko: Formálne jazyky a regulárne výrazy, konečné automaty (deterministické a nedeterministické, lexikálna analýza a jej úloha v prekladačoch, prehľad existujúcich nástrojov na lexikálnu analýzu)
- Návrh lexikálneho analyzátora: špecifikácia tokenov pomocou regulárnych výrazov, prevody regulárnych výrazov na konečné automaty, architektúra a modulárnosť riešenia.
- Implementácia: popis implementačných detailov, spracovanie vstupného textu, algoritmus prechodu autoatom a rozpoznávanie tokenov, podpora základných príkazov imperatívneho jazyka, vyhodnocovanie základných matematických výrazov
- Testovanie a vyhodnotenie: testovacie scenáre a použité vstupné dáta, porovnanie s existujúcimi nástrojmi, výkonnosť a spoľahlivosť analyzátora, možnosti rozšírenia a vylepšenia
- Záver: Zhrnutie dosiahnutých výsledkov, splnenie cieľov práce, návrhy na ďalší výskum a rozvoj

Meno a pracovisko vedúceho BP: Mgr. Peter Novotný, PhD., KST, ŽU
Meno a pracovisko tutora BP:

garant štud. programu
(dátum a podpis)

Zadanie zaregistrované dňa 31. 10. 2025 pod číslom 3343/2025 podpis _____

Čestné vyhlásenie

Vyhlasujem, že som zadanú bakalársku prácu vypracoval samostatne, pod odborným vedením vedúceho práce/školiteľa a používal som len literatúru uvedenú v práci.

Vyhlasujem, že som pri vypracúvaní predloženej záverečnej práce využil nástroje umelej inteligencie ChatGPT-5.3 výhradne na účely gramatickej korektúry textu, štylistickej úpravy, generovania štruktúry práce.

Potvrdzujem, že výsledný text práce je mojim autorským dielom, za jeho odbornú náplň nesiem plnú zodpovednosť a žiadna časť práce nebola prevzatá z výstupu umelej inteligencie bez kritického zhodnotenia a riadneho citovania v súlade s etickými princípmi.

Žilina 3. marca 2026

podpis

Poďakovanie

V prvom rade by som rád poďakoval školiteľovi bakalárskej práce, Mgr. Petrovi Novotnému, PhD., za pomoc pri vypracovaní bakalárskej práce a ochotu zodpovedať na každú otázku. Osobitné poďakovanie patrí mojej rodine za neustálu podporu popri štúdiu. V neposlednom rade patrí veľké ďakujem spolužiakom na Fakulte riadenia a informatiky za vzájomnú pomoc pri prekonávaní rôznych prekážok počas štúdia.

Abstrakt

GUBÁNY, Jakub: Využitie regulárnych výrazov a konečných automatov pri tvorbe lexikálneho analyzátora pre základný imperatívny jazyk. [Bakalárska práca] / Jakub Gubány. – Žilinská univerzita v Žiline. Fakulta riadenia a informatiky.

Práca sa zaoberá návrhom a implementáciou lexikálneho analyzátora pre jednoduchý imperatívny programovací jazyk s využitím regulárnych výrazov a konečných automatov. Teoretická časť predstavuje základné pojmy teórie formálnych jazykov, regulárne jazyky, deterministické a nedeterministické konečné automaty a ich vzájomné prevody, pričom dôraz je kladený na ich praktické využitie v procese lexikálnej analýzy. Praktická časť sa zameriava na formálnu špecifikáciu tokenov pomocou regulárnych výrazov, konštrukciu nedeterministického a deterministického konečného automatu, konštrukčným algoritmom a využitím deterministického konečného automatu nad vstupným reťazcom. Výsledkom práce je modulárny a rozšíriteľný lexikálny analyzátor implementovaný v jazyku Java, schopný identifikovať lexikálne jednotky definované používateľom na základe tokenizačných pravidiel a nakonfigurovanej abecedy. Implementované riešenie je porovnané s vybranými existujúcimi nástrojmi na lexikálnu analýzu z hľadiska prehľadnosti, modularity a efektívnosti. Práca súčasne poskytuje praktické overenie teoretických poznatkov o konečných automatoch. Navrhnutý analyzátor môže zároveň slúžiť ako základ pre tvorbu jednoduchých doménovo špecifických jazykov alebo ďalších nástrojov na spracovanie textových formátov.

Kľúčové slová: lexikálna analýza, regulárne výrazy, konečné automaty, nedeterministický a deterministický konečný automat, token, prekladač, imperatívny jazyk.

Abstract

GUBÁNY, Jakub: Application of Regular Expressions and Finite Automata in the Development of a Lexical Analyzer for a Basic Imperative Language. [Bachelor's Thesis] / Jakub Gubány. – University of Žilina. Faculty of Management Science and Informatics.

The thesis focuses on the design and implementation of a lexical analyzer for a simple imperative programming language using regular expressions and finite automata. The theoretical part introduces the fundamental concepts of formal language theory, regular languages, deterministic and nondeterministic finite automata, and their mutual transformations, with an emphasis on their practical application in the process of lexical analysis. The practical part focuses on the formal specification of tokens using regular expressions, the construction of nondeterministic and deterministic finite automata, the underlying construction algorithms, and the application of a deterministic finite automaton to process the input string. The result of the thesis is a modular and extensible lexical analyzer implemented in Java, capable of identifying lexical units defined by the user based on tokenization rules and a configured input alphabet. The implemented solution is compared with selected existing tools for lexical analysis in terms of clarity, modularity, and efficiency. The thesis also provides a practical validation of theoretical concepts related to finite automata. The proposed analyzer can serve as a foundation for the development of simple domain-specific languages or other tools for processing textual data formats.

Keywords: lexical analysis, regular expressions, finite automata, deterministic and nondeterministic finite automaton, token, compiler, imperative language

Obsah

Úvod 11

1 Formálne jazyky a regulárne výrazy 12
- 1.1 Abeceda, slovo a jazyk 12
- 1.2 Regulárne jazyky 12
- 1.3 Regulárne výrazy 13

2 Ako rozpoznávať regulárne jazyky 14
- 2.1 Rozpoznávanie jazyka konečným automatom 14
- 2.2 Nedeterministický konečný automat (NFA) 15
- 2.3 Deterministický konečný automat (DFA) 16
- 2.4 Prevod regulárneho výrazu na NFA 17
- 2.5 Prevod NFA na DFA 17

3 Lexikálna analýza v prekladačoch 18
- 3.1 Úloha a význam lexikálnej analýzy 18
- 3.2 Tokeny, lexémy a regulárne výrazy 19
- 3.3 Generátory lexikálnych analyzátorov 19
  - 3.3.1 Flex 19
  - 3.3.2 ANTLR 20
- 3.4 Lexikálne chyby a ich spracovanie 21

4 Návrh vlastného lexikálneho analyzátora 21
- 4.1 Špecifikácia imperatívneho jazyka 21
  - 4.1.1 Príklad vstupného programu 22
- 4.2 Architektúra riešenia 22

5 Implementácia 23
- 5.1 Definovanie tokenizačných pravidiel 23
- 5.2 Implementácia regulárnych výrazov 24
  - 5.2.1 Spracovanie regulárneho výrazu 24
  - 5.2.2 Explicitné zreťazenie 24
  - 5.2.3 Postfixová forma 25
- 5.3 Implementácia NFA 25
  - 5.3.1 Reprezentácia NFA 25
  - 5.3.2 Konštrukcia NFA 26
  - 5.3.3 Pripravenosť na prevod do DFA 26
- 5.4 Implementácia DFA 27
  - 5.4.1 Reprezentácia DFA 27
  - 5.4.2 Konštrukcia DFA 27
  - 5.4.3 Výsledná podoba DFA 28
- 5.5 Implementácia lexera 28
  - 5.5.1 Konfigurácia abecedy 28
  - 5.5.2 Princíp najdlhšej zhody 29
  - 5.5.3 Algoritmus spracovania vstupu 29
- 5.6 Generovanie tokenov 30

6 Testovanie a vyhodnotenie 30
- 6.1 Testovacie scenáre 30

6.1.1 Imperatívny jazyk 30
6.1.2 Aritmetické výrazy 31
6.1.3 Formát CSV 31
6.1.4 Chybné vstupy 31
6.2 Výkonnostné testy 32
6.2.1 Experiment 32
6.3 Porovnanie s Flex a ANTLR 33
7 Diskusia výsledkov 34
8 Možnosti využitia a rozšírenia 35
8.1 Využitie pri tvorbe vlastného programovacieho jazyka 35
8.2 Využitie pri spracovaní doménovo špecifických jazykov 35
8.3 Využitie mimo kompilátorov 36
8.4 Možnosti rozšírenia 36
8.4.1 Paralelizácia spracovania 36
Záver 37

Zoznam obrázkov

Obrázok 2.1 – NFA nad abecedou, ktorý prijíma všetky slová končiace slovom abb ...16
Obrázok 2.2 – DFA nad abecedou, ktorý prijíma všetky slová končiace slovom abb ...17
Obrátok 3.1 – Spracovania vstupu v nástroji ANTLR 20
Obrázok 5.1 – Obrazovka s definíciami tokenizačných pravidiel 23
Obrázok 5.2 – Konfiguračné okno vstupnej abecedy 28
Obrázok 5.3 – Výpis tokenov a lexikálnych chýb 30
Obrázok 6.1 – Okno s prehľadom výkonnostných metrík 32
Obrázok 6.2 – Graf závislosti času od spracovania dĺžky vstupu 33

Zoznam tabuliek

Tabuľka 6.1 – Meranie času spracovania vstupu 32

Zoznam obrázkov v prílohách

Obrázok 3 – Hlavné okno aplikácie 22
Obrázok 4 – Tabuľka pozícií 23
Obrázok 5 – Nastavenie abecedy 24
Obrázok 6 – Výkonnostné metriky 24
Obrázok 7 – UML diagram tried jadra lexikálneho analyzátora 26
Obrázok 8 – UML diagram tried používateľského rozhrania 27

Zoznam tabuliek v prílohách

Tabuľka 1 – Požiadavky pre spustenie aplikácie 21
Tabuľka 2 – Funkcie jednotlivých tlačidiel 22
Tabuľka 3 – Význam jednotlivých metrík 25
Tabuľka 4 – Význam jednotlivých polí v konfigurácii pravidiel tokenov 25

Zoznam skratiek

DFA	**Deterministic Finite Automaton**	**Deterministický konečný automat**
NFA	**Nondeterministic Finite Automaton**	**Nedeterministický konečný automat**
AST	**Abstract Syntax Tree**	**Abstraktný syntaktický strom**
CSV	**Comma-Separated Values**	**Hodnoty oddelené čiarkou**
JVM	**Java Virtual Machine**	**Virtuálny stroj jazyka Java**
JIT	**Just-In-Time (Compilation)**	**Just-In-Time kompilácia**
DSL	**Domain-Specific Language**	**Doménovo špecifický jazyk**

Zoznam symbolov

Symbol	Jednotka	Význam symbolu
Σ	Abeceda	Konečná množina symbolov
Q	Množina	Množina stavov
δ	Funkcia	Prechodová funkcia
ε	Slovo	Prázdne slovo
q_0	Stav	Počiatočný stav
F	Množina	Množina akceptačných stavov
Σ^*	Jazyk	Množina všetkých slov nad abecedou Σ
$\emptyset$	Jazyk	Prázdny jazyk
.	Operácia	Zreťazenie slov
$\cup$	Operácia	Zjednotenie jazykov
*	Operácia	Kleeneho uzáver
{a}	Jazyk	Jazyk obsahujúci jeden symbol

ÚVOD

Spracovanie programovacích jazykov predstavuje jednu zo základných oblastí informatiky, ktorá sa zaoberá prekladom a interpretáciou zdrojového kódu do podoby vykonateľnej počítačom. Každý programovací jazyk, bez ohľadu na svoju zložitosť, prechádza počas spracovania viacerými fázami, medzi ktoré patrí lexikálna analýza, syntaktická analýza a sémantické spracovanie. Tieto fázy spolu vytvárajú jadro kompilátorov a interpreterov.

Prvou a zároveň nevyhnutnou etapou tohto procesu je lexikálna analýza. Jej úlohou je transformovať vstupný reťazec znakov predstavujúci zdrojový kód na postupnosť základných jednotiek – tokenov. Tokeny reprezentujú syntakticky významné prvky jazyka, ako sú kľúčové slová, identifikátory, literály alebo operátory. Správna a efektívna implementácia lexikálnej analýzy je kľúčová pre ďalšie fázy spracovania programu.

Teoretický základ lexikálnej analýzy je úzko spätý s teóriou formálnych jazykov, konkrétne s regulárnymi jazykmi a konečnými automatmi. Regulárne výrazy umožňujú formálne špecifikovať množinu povolených lexikálnych jednotiek, zatiaľ čo konečné automaty poskytujú efektívny mechanizmus na ich rozpoznávanie. Vďaka ich matematickej presnosti a algoritmickej realizovateľnosti predstavujú prirodzený most medzi teóriou a praktickou implementáciou.

1 FORMÁLNE JAZYKY A REGULÁRNE VÝRAZY

Formálne jazyky predstavujú matematický aparát na opis množín reťazcov nad konečnou abecedou. V informatike nachádzajú uplatnenie najmä pri návrhu a implementácii prekladačov, kde umožňujú presne špecifikovať štruktúru a význam programovacieho jazyka [1]. V kontexte lexikálnej analýzy majú osobitný význam regulárne jazyky, pretože ich možno vyjadriť regulárnymi výrazmi a zároveň efektívne rozpoznávať pomocou konečných automatov [2][3].

Táto kapitola predstavuje základné pojmy z oblasti formálnych jazykov a regulárnych výrazov, ktoré tvoria teoretický základ pre návrh lexikálneho analyzátora.

1.1 Abeceda, slovo a jazyk

Základným pojmom formálnych jazykov je **abeceda**. Abeceda Σ je konečná neprázdna množina symbolov. Symboly abecedy predstavujú elementárne jednotky, z ktorých sa skladajú slová.

Slovo nad abecedou Σ je konečná postupnosť symbolov z tejto abecedy. Prázdne slovo, ktoré neobsahuje žiadne symboly, sa označuje ε. Dĺžka slova w sa označuje $|w|$, pričom pre prázdne slovo platí $|\varepsilon| = 0$ [1].

Množinu všetkých slov dĺžky n nad abecedou Σ označujeme Σ^n. Zjednotením týchto množín vzniká množina všetkých slov nad abecedou Σ, označovaná Σ^* [1].

Zo slov možno vytvárať nové slová pomocou operácie **zreťazenia**. Ak sú u a v slová, potom ich zreťazenie uv je slovo vzniknuté pripojením slova v za slovo u.

Dôležitou operáciou je aj **Kleeneho uzáver**. Pre jazyk L definujeme L^* ako množinu všetkých slov, ktoré vzniknú zreťazením ľubovoľného (vrátane nulového) počtu slov z jazyka L. Táto operácia umožňuje popis opakovania vzorov v jazyku [1].

Formálny jazyk nad abecedou Σ je ľubovoľná podmnožina množiny Σ^*. Inými slovami, jazyk predstavuje množinu slov, ktoré spĺňajú určité pravidlá alebo vlastnosti.

Vstupný program možno z pohľadu lexikálnej analýzy rozumieť ako jedno dlhé slovo nad vhodne definovanou abecedou. Úlohou lexikálneho analyzátora je toto slovo rozdeliť na menšie významové jednotky – tokeny [2].

1.2 Regulárne jazyky

Pri práci s formálnymi jazykmi je vhodné rozlišovať medzi rôznymi triedami jazykov podľa ich vlastností. Jednou z najvýznamnejších tried sú jazyky, ktoré možno popísať pomocou regulárnych výrazov (*kapitola 1, časť 1.3*) [2][3]. Takéto jazyky sa formálne označujú ako **regulárne jazyky.** Sú charakteristické jednoduchou štruktúrou, ktorá umožňuje ich následné, efektívne spracovanie.

Regulárne jazyky možno definovať ako najmenšiu triedu jazykov nad abecedou Σ, ktorá spĺňa nasledujúce podmienky:

1. obsahuje prázdny jazyk $\emptyset$,
2. obsahuje jazyk $\{\varepsilon\}$,
3. obsahuje jazyk $\{a\}$ pre každý symbol $a \in \Sigma$,
4. je uzavretá na operácie zjednotenia, zreťazenia a Kleeneho uzáveru [1][3].

Vhodné by bolo ukázať aplikáciu vyššie spomenutých pravidiel v prípade abecedy Σ, ktorá obsahuje práve dva symboly a, b. Napríklad jazyk $\{a\}$ je regulárny a aj jazyk $\{b\}$ je regulárny. Ich zjednotením vznikne jazyk $\{a, b\}$. Zreťazením jazyka $\{b\}$ so sebou samým vznikne jazyk $\{bb\}$. Aplikovaním Kleeneho uzáveru nad jazykom $\{ab\}$ získame jazyk $\{\varepsilon, ab, abab, ababab, \dots\}$.

Keďže trieda regulárnych jazykov je definovaná ako uzavretá na základné operácie, možno z tejto definície odvodiť aj ďalšie vlastnosti. Regulárne jazyky sú napríklad uzavreté na prienik, doplnok alebo rozdiel jazykov [3].

V kontexte programovacích jazykov možno mnohé lexikálne jednotky, ako napríklad identifikátory, číselné literály alebo operátory, špecifikovať pomocou regulárnych výrazov, ktoré definujú regulárne jazyky [2]. To umožňuje ich efektívne rozpoznávanie pomocou konečných automatov.

1.3 Regulárne výrazy

Regulárne výrazy predstavujú formálny spôsob zápisu množín slov nad danou abecedou.

Každý regulárny výraz definuje určitý jazyk. Úplným základom regulárnych výrazov tvoria prípady, medzi ktoré patrí prázdny jazyk $\emptyset$, jazyk obsahujúci prázdne slovo ε a jednotlivé symboly abecedy. Z týchto základov možno vytvárať zložitejšie výrazy pomocou operácie **alternácie**, **zreťazenia** a **Kleeneho uzáveru**.

V regulárnom výraze $(a \mid b)c^*$ alternácia $(a \mid b)$ umožňuje výber medzi znakom a a b, ďalej Kleeneho uzáver $*$ dovoľuje opakovať znak c ľubovoľný počet krát a zreťazenie spája v tomto regulárnom výraze výsledok alternácie a Kleeneho uzáveru. Príkladom by mohlo byť slovo $bccc$ alebo ac, ktoré je, už ako bolo spomenuté, prvkom konkrétneho regulárneho jazyka, ktorý bol definovaný konkrétnym regulárnym výrazom.

Z praxe sú známymi tokenmi napríklad kľúčové slová (int, double), ktoré zohrávajú podstatnú úlohu vo väčšine programovacích jazykov. Tieto tokeny možno špecifikovať konkrétnym regulárnym výrazom a ten určuje množinu všetkých slov patriacich do danej kategórie. Táto množina tvorí regulárny jazyk.

2 AKO ROZPOZNÁVAŤ REGULÁRNE JAZYKY

V predchádzajúcej kapitole bolo naznačené, akým spôsobom sa pomocou regulárnych výrazov špecifikujú regulárne jazyky. Doposiaľ však nebolo vysvetlené, akým mechanizmom sa následne rozhoduje o tom, či konkrétne slovo do takto definovaného jazyka patrí.

Rozpoznávanie regulárnych jazykov predstavuje základný problém teórie formálnych jazykov a zároveň tvorí základ lexikálnej analýzy v prekladačoch. Na tento účel sa využívajú **konečné automaty**. Taktiež je vhodné povedať, že regulárne jazyky sú ekvivalentné konečným automatom, t. j. každý regulárny jazyk možno rozpoznať konečným automatom a naopak [2][3].

Konečný automat je formálne definovaný ako pätica

$$A = (Q, \Sigma, \delta, q_0, F),$$

kde Q je konečná množina stavov, Σ je vstupná abeceda, δ je prechodová funkcia, $q_0 \in Q$ je počiatočný stav a $F \subseteq Q$ je množina akceptačných stavov [1][2].

V závislosti od vlastností prechodovej funkcie rozlišujeme **deterministické (DFA)** a **nedeterministické (NFA)** konečné automaty.

2.1 Rozpoznávanie jazyka konečným automatom

Konečný automat spracováva vstupné slovo postupným čítaním jeho symbolov. Spracovanie začína v počiatočnom stave a po prečítaní každého symbolu sa automat presunie do ďalšieho stavu podľa definovaných prechodov. Týmto spôsobom vzniká postupnosť stavov, ktorá zodpovedá spracovaniu celého vstupného slova.

Takúto postupnosť stavov možno chápať ako cestu v grafe automatu. Každý prechod na tejto ceste je určený aktuálnym stavom a vstupným symbolom, ktorý je práve spracovávaný.

Vstupné slovo je prijaté práve vtedy, ak po spracovaní všetkých jeho symbolov existuje cesta z počiatočného stavu do niektorého z akceptačných stavov. Ak takáto cesta neexistuje, slovo je automatom odmietnuté.

Tento princíp tvorí základ rozpoznávania regulárnych jazykov pomocou konečných automatov a je využívaný napríklad v lexikálnej analýze, kde automat identifikuje platné tokeny vo vstupnom texte.

Štruktúra konečného automatu, teda množina jeho stavov a prechodov, nie je náhodná, ale je odvodená z regulárneho výrazu. Existujú systematické postupy (*kapitola 2, časť 2.4*), ktoré umožňujú previesť regulárny výraz na konečný automat tak, aby rozpoznával presne ten istý jazyk.

2.2 Nedeterministický konečný automat (NFA)

Nedeterministický konečný automat (*ďalej len NFA*) predstavuje špecifický model konečného automatu. Automat sa počas spracovávania nachádza v určitom stave, pričom takýto stav reprezentuje čiastočný výsledok spracovania slova.

V prípade NFA pre konkrétny stav a vstupný symbol nemusí byť určený jednoznačný nasledujúci stav. Na rozdiel od deterministického konečného automatu umožňuje existenciu viacerých možných prechodov pre rovnakú dvojicu (stav, symbol). Automat sa teda môže po spracovaní jedného symbolu nachádzať aj v niekoľkých stavoch súčasne [1].

Formálne je prechodová funkcia NFA definovaná ako:

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q$$

Počas spracovania vstupného slova môže nastať situácia, keď je potrebné prejsť medzi jednotlivými stavmi automatu bez potreby spracovania ďalšieho symbolu. Napríklad pri rozpoznávaní jazyka definovaného regulárnym výrazom, konkrétne alternáciou $a \mid b$ má automat na začiatku „vybrať“ či bude spracovávať symbol a alebo b. Táto voľba automat „nestojí“ prečítanie ďalšieho vstupného symbolu, ale iba prechod do príslušnej vetvy automatu.

Pre tento účel slúžia tzv. **ε-prechody**, ktoré umožňujú zmenu stavu bez spotrebovania vstupného symbolu. Tieto prechody výrazne zjednodušujú konštrukciu automatov, najmä pri systematickom prevode regulárnych výrazov na konečné automaty, napríklad pomocou Thompsonovej konštrukcie (*kapitola 2, časť 2.4*) [2].

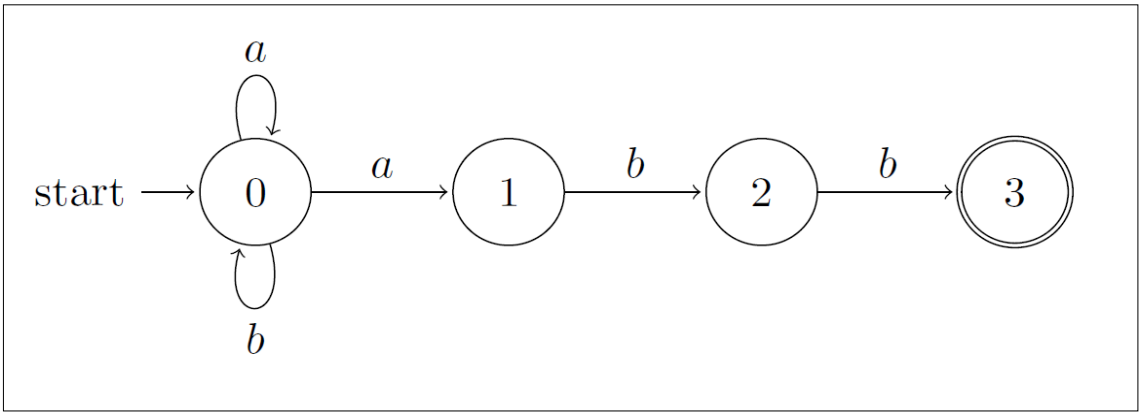
Z hľadiska spracovania vstupu NFA paralelne skúma všetky možné cesty. V praxi to znamená, že ak existuje aspoň jedna cesta vedúca do akceptačného stavu, vstupný reťazec je prijatý.

Na *obrázku 2.1.* uvažujme NFA nad abecedou $\Sigma = \{a, b\}$, ktorý rozpoznáva jazyk všetkých reťazcov končiacich podreťazcom abb.

Automat obsahuje niekoľko stavov, pričom jeden z nich je počiatočný a jeden je akceptačný. Z počiatočného stavu vedú prechody pre symboly a aj b späť do toho istého stavu, čím je umožnené spracovanie ľubovoľného prefixu vstupného reťazca. Zároveň však existuje prechod, ktorý reprezentuje začiatok rozpoznávania požadovanej koncovky abb.

Po prečítaní symbolu a automat prejde do ďalšieho stavu, z ktorého nasleduje prechod na symbol b a následne ďalší prechod na symbol b, ktorý vedie do akceptačného stavu. Vďaka nedeterministickej povahe automatu môže systém „uhádnuť“, kedy sa začína sledovaná sekvencia abb, a paralelne pokračovať aj v spracovaní iných možností.

Tento príklad ilustruje základný princíp fungovania NFA – automat môže mať viacero možných výpočtových ciest, pričom vstupný reťazec je prijatý, ak existuje aspoň jedna cesta vedúca do akceptačného stavu [1].



Obrázok 2.1 – NFA, ktorý prijíma všetky slová končiace slovom abb

Zdroj: (vlastné spracovanie)

2.3 Deterministický konečný automat (DFA)

Na rozdiel od nedeterministického automatu je v deterministickom konečnom automate (*ďalej len DFA*) pre každý stav a vstupný symbol definovaný **najviac jeden nasledujúci stav.**

Prechodová funkcia má tvar:

$$\delta : Q \times \Sigma \rightarrow Q$$

To znamená, že spracovanie vstupného reťazca je jednoznačné a v každom kroku sa automat nachádza práve v jednom stave.

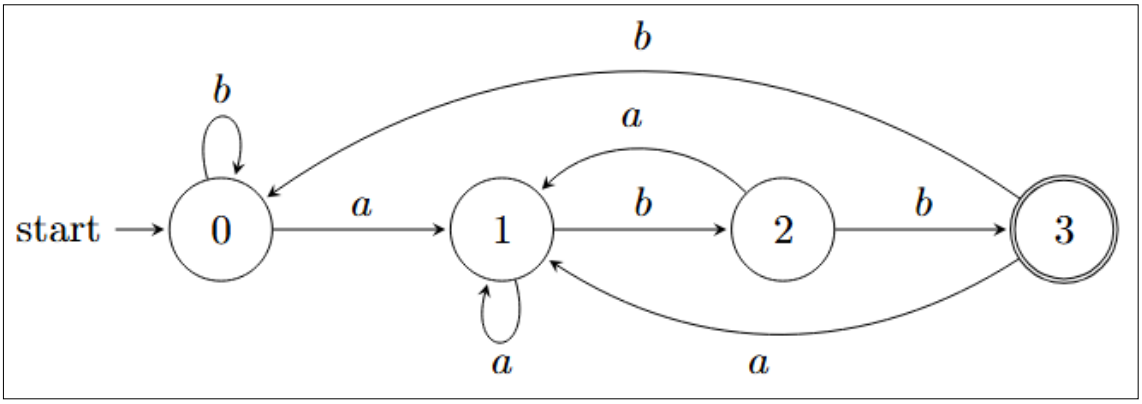
Deterministické automaty neobsahujú ε-prechody a pre každú kombináciu stavu a vstupného symbolu je prechod buď definovaný, alebo vedie do neakceptačného stavu. Vďaka tejto jednoznačnosti sú DFA vhodné pre praktickú implementáciu, napríklad v lexikálnych analyzátoroch, kde je potrebné efektívne spracovanie vstupu [2][3].

Na obrázku 2.2 je znázornený DFA nad abecedou $\Sigma = \{a, b\}$, ktorý rozpoznáva jazyk všetkých reťazcov končiacich podreťazcom abb.

Automat obsahuje konečný počet stavov reprezentujúcich postupné rozpoznávanie sledovanej sekvencie. Pri spracovaní symbolov vstupu sa automat presúva medzi stavmi podľa toho, do akej miery sa aktuálna koncovka vstupného reťazca zhoduje so vzorom abb.

Po prečítaní symbolu a automat prejde do stavu reprezentujúceho začiatok potenciálnej zhody. Následne po prečítaní symbolu b prejde do ďalšieho stavu a po druhom symbole b dosiahne akceptačný stav. Ak vstup obsahuje symboly, ktoré narušia očakávanú sekvenciu, automat sa vráti do vhodného predchádzajúceho stavu, ktorý zodpovedá najdlhšiemu možnému prefixu vzoru.

Tento mechanizmus zabezpečuje, že automat vždy jednoznačne určuje svoj nasledujúci stav a dokáže efektívne rozpoznávať požadovaný jazyk bez potreby skúmania viacerých paralelných ciest, ako je tomu pri NFA [1].



Obrázok 2.2 – DFA, ktorý prijíma všetky slová končiace slovom abb

Zdroj: (vlastné spracovanie)

2.4 Prevod regulárneho výrazu na NFA

Pre praktické využitie je dôležitý prevod regulárneho výrazu na konečný automat. Jednou z najpoužívanejších metód tohto prevodu je **Thompsonova konštrukcia**, ktorá poskytuje systematický postup na vytvorenie nedeterministického konečného automatu z regulárneho výrazu [2][3].

Thompsonova konštrukcia je založená na rozklade regulárneho výrazu na menšie časti, pre ktoré sa postupne vytvárajú jednoduché automaty, tzv. fragmenty. Tieto fragmenty sa následne skladajú do jedného výsledného automatu tak, aby rozpoznával rovnaký formálny jazyk ako pôvodný regulárny výraz [2][3].

Pre jednotlivý symbol abecedy sa vytvorí automat s počiatočným a koncovým stavom, medzi ktorými vedie prechod označený daným symbolom. Podobne je spracované aj prázdne slovo ε, ktoré je reprezentované prechodom bez spotreby vstupu. Tieto jednoduché automaty tvoria základ pre konštrukciu zložitejších výrazov [1].

Pri vytváraní komplexnejších automatov sa využívajú operácie regulárnych výrazov. Pri zreťazení sa automat pre prvý výraz spojí s automatom pre druhý výraz pomocou ε-prechodu (*kapitola 2, časť 2.2*). Pri alternácii sa vytvorí nový počiatočný stav, z ktorého vedú ε-prechody do oboch vetiev, a ich koncové stavy sa následne zjednotia. V prípade Kleeneho uzáveru sa pridávajú prechody umožňujúce opakovanie výrazu alebo jeho vynechanie [2][3].

Významnou vlastnosťou tejto konštrukcie je využitie ε-prechodov, vďaka nim môže byť výsledný automat nedeterministický.

Výsledkom Thompsonovej konštrukcie je nedeterministický konečný automat, ktorý možno následne previesť na deterministický konečný automat.

2.5 Prevod NFA na DFA

Aj keď sa NFA a DFA líšia spôsobom spracovania vstupu, z hľadiska schopnosti rozpoznávania formálneho jazyka sú ekvivalentné [1][2].

Prevod NFA na DFA sa realizuje pomocou tzv. **konštrukcie podmnožín**. Konštrukcia spočíva v tom, že každý stav DFA reprezentuje množinu stavov pôvodného NFA. Inými

slovami, DFA reprezentuje všetky možné stavy, v ktorých sa môže NFA nachádzať po spracovaní časti vstupu, a pracuje s nimi ako s jedným celkom [2][3].

Proces konštrukcie začína z počiatočného stavu, ktorý zodpovedá množine obsahujúcej počiatočný stav NFA spolu so všetkými stavmi dosiahnuteľnými pomocou ε-prechodov. Následne sa pre každý vstupný symbol určuje, do akej množiny stavov sa môže automat dostať, pričom sa opäť zohľadňujú ε-prechody. Tento postup sa opakuje, kým nie sú spracované všetky dosiahnuteľné množiny stavov, ktoré následne tvoria stavy výsledného DFA [2].

Výsledný deterministický automat rozpoznáva rovnaký jazyk ako pôvodný nedeterministický automat, pričom každému vstupnému symbolu zodpovedá práve jeden prechod z každého stavu. Táto vlastnosť zabezpečuje jednoznačné a efektívne spracovanie vstupu, čo je zásadné najmä pri implementácii lexikálnych analyzátorov [1][2].

3 LEXIKÁLNA ANALÝZA V PREKLADAČOCH

Lexikálna analýza predstavuje prvú fázu prekladu programu, ktorej úlohou je transformovať vstupný zdrojový text na postupnosť lexikálnych jednotiek – tokenov. Táto fáza sa často označuje aj ako skenovanie [2].

Lexikálny analyzátor postupne spracúva vstupný reťazec symbolov a identifikuje v ňom lexémy, teda konkrétne výskyty jazykových jednotiek. Každej lexéme priraďuje príslušný token, ktorý reprezentuje jej kategóriu (napr. identifikátor, kľúčové slovo, operátor alebo literál). Výstupom lexikálnej analýzy je sekvencia tokenov, ktorá slúži ako vstup pre syntaktickú analýzu [2].

Okrem samotného rozpoznávania tokenov lexikálny analyzátor zabezpečuje aj ďalšie úlohy, ako je ignorovanie bielych znakov a komentárov, evidencia pozície vo vstupnom texte (riadok, stĺpec) a detekcia lexikálnych chýb. Týmto spôsobom sa zjednodušuje práca nasledujúcich fáz prekladu a zvyšuje sa robustnosť celého prekladača [3].

3.1 Úloha a význam lexikálnej analýzy

Hlavným cieľom lexikálnej analýzy je prevod nízkoúrovňovej reprezentácie programu (sekvencie symbolov) na vyššiu abstraktnú úroveň (tokeny), s ktorou ďalej pracuje syntaktická analýza. Tento krok výrazne zjednodušuje návrh parsera, ktorý sa už nemusí zaoberať jednotlivými znakmi, ale pracuje s významovými jednotkami [2].

Dôležitým aspektom lexikálnej analýzy je aj oddelenie jednotlivých fáz prekladu. Lexikálny analyzátor rieši problémy spojené s rozpoznávaním lexém, zatiaľ čo syntaktický analyzátor sa sústreďuje na štruktúru programu. Takáto modularita umožňuje jednoduchšiu implementáciu aj údržbu prekladača [3].

Lexikálny analyzátor často spolupracuje so symbolickou tabuľkou, do ktorej ukladá informácie o identifikátoroch a literáloch. Pri rozpoznaní identifikátora môže napríklad vložiť jeho záznam do tabuľky alebo získať referenciu na už existujúci záznam. Symbolická tabuľka je následne využívaná v ďalších fázach prekladu, najmä pri sémantickej analýze [2][4].

3.2 Tokeny, lexémy a regulárne výrazy

Token predstavuje abstraktnú kategóriu lexikálnych jednotiek, zatiaľ čo lexéma je konkrétny reťazec znakov vo vstupnom texte, ktorý tejto kategórii zodpovedá. Napríklad reťazec *value* môže byť lexémou tokenu typu identifikátor, zatiaľ čo reťazec *if* predstavuje lexému tokenu typu kľúčové slovo.

Na formálnu špecifikáciu tokenov sa využívajú regulárne výrazy. Regulárne výrazy poskytujú kompaktný a presný spôsob popisu množín reťazcov a umožňujú definovať základné lexikálne jednotky jazyka, ako sú identifikátory, čísla alebo operátory [1].

Formálne sú regulárne výrazy ekvivalentné konečným automatom, čo znamená, že každý regulárny výraz je možné transformovať na konečný automat, ktorý rozpoznáva rovnaký jazyk. Táto vlastnosť je kľúčová pre implementáciu lexikálnych analyzátorov, keďže umožňuje efektívne spracovanie vstupu [1].

3.3 Generátory lexikálnych analyzátorov

Generátory lexikálnych analyzátorov umožňujú definovať tokeny pomocou regulárnych výrazov a následne automaticky generujú efektívny skener. Vygenerovaný analyzátor implementuje konečný automat a zabezpečuje rozpoznávanie lexém vo vstupnom texte.

Výhodou tohto prístupu je, že programátor sa sústreďuje na definovanie pravidiel, zatiaľ čo implementačné detaily sú riešené nástrojom. Tým sa znižuje komplexnosť vývoja a zvyšuje spoľahlivosť výsledného analyzátora [2].

3.3.1 Flex

Flex (Fast Lexical Analyzer Generator) je nástroj na generovanie lexikálnych analyzátorov, ktorý vychádza z historického nástroja Lex. Používateľ v ňom špecifikuje množinu pravidiel vo forme regulárnych výrazov, ku ktorým sú priradené akcie vykonávané pri rozpoznaní zodpovedajúcich lexém.

Špecifikácia vo Flexe je rozdelená do viacerých častí, pričom jadrom je sekcia obsahujúca dvojice „regulárny výraz – akcia“. Pri spracovaní vstupu generovaný analyzátor vyhľadáva najdlhšiu lexému, ktorá zodpovedá niektorému z definovaných pravidiel. V prípade viacerých možných zhôd sa uplatňuje poradie pravidiel v špecifikácii, pričom sa vyberá prvé zodpovedajúce pravidlo [5].

Na základe tejto špecifikácie Flex generuje zdrojový kód analyzátora, najčastejšie v jazyku C. Vygenerovaný analyzátor je implementovaný ako deterministický konečný automat, ktorý efektívne spracúva vstupný text po jednotlivých znakoch [2].

Flex je široko používaný najmä v tradičných kompilátorových nástrojoch a systémoch založených na jazyku C.

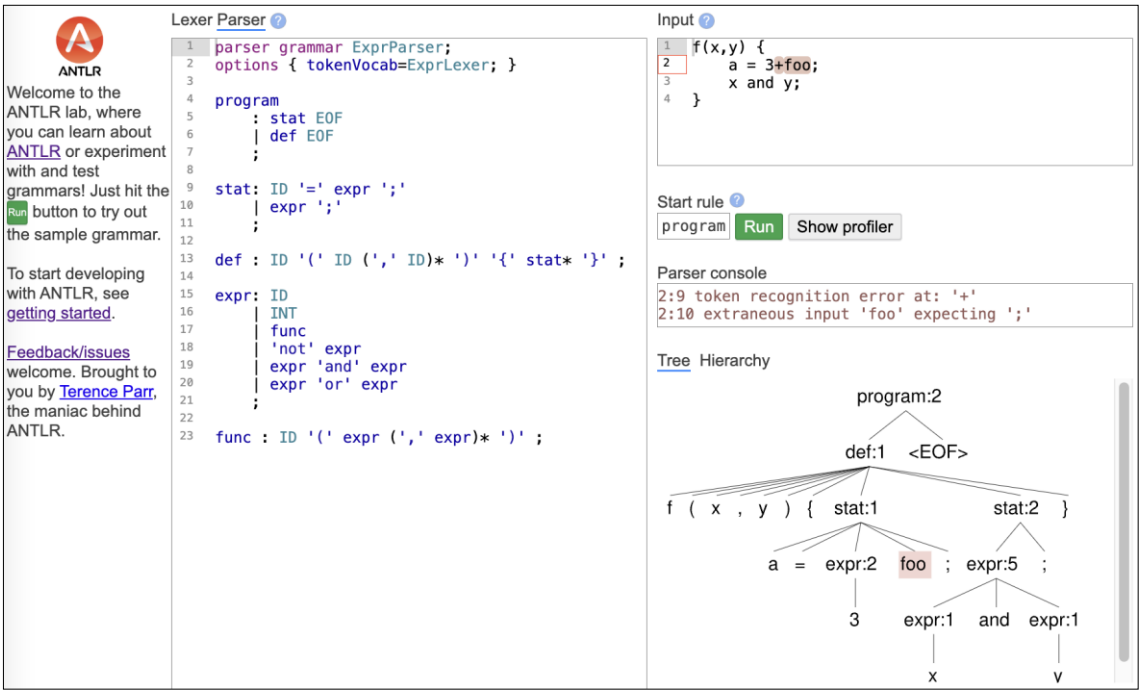
3.3.2 ANTLR

ANTLR (Another Tool for Language Recognition) predstavuje moderný nástroj na generovanie analyzátorov, ktorý umožňuje definovať lexikálnu aj syntaktickú štruktúru jazyka v jednom gramatickom súbore.

Na rozdiel od nástrojov typu Lex/Flex, ktoré pracujú primárne s regulárnymi výrazmi, ANTLR využíva rozšírené bezkontextové gramatiky. Lexer v ANTLR je generovaný na základe lexikálnych pravidiel definovaných v gramatike a rozpoznáva tokeny podľa princípu najdlhšej zhody, pričom pri konfliktoch zohráva úlohu aj poradie pravidiel [6].

Významnou vlastnosťou ANTLR je generovanie kompletného analyzátora, ktorý zahŕňa lexikálnu aj syntaktickú analýzu. Nástroj zároveň automaticky vytvára štruktúry pre reprezentáciu syntaktického stromu a poskytuje mechanizmy na jeho spracovanie, napríklad pomocou návštevníckych vzorov (*visitor pattern*) [3][6].

ANTLR podporuje generovanie kódu vo viacerých programovacích jazykoch, ako sú Java, C#, Python alebo JavaScript, a je často využívaný pri návrhu programovacích jazykov alebo doménovo špecifických jazykov.



Obrátok 3.1 – Spracovania vstupu v nástroji ANTLR

Zdroj: (https://www.antlr.org)

3.4 Lexikálne chyby a ich spracovanie

Lexikálny analyzátor je zodpovedný aj za detekciu chýb na úrovni jednotlivých znakov alebo lexém. Medzi typické lexikálne chyby patria napríklad výskyt nepovolených znakov, nesprávne zapísané literály alebo neukončené reťazce.

Pri detekcii chyby môže analyzátor:

- preskočiť problematický znak,
- pokúsiť sa o jednoduchú opravu,
- alebo zaznamenať chybu a pokračovať v analýze.

Cieľom je umožniť pokračovanie spracovania vstupu a identifikovať čo najviac chýb v jednom prechode, čo je dôležité najmä z pohľadu používateľa prekladača [2].

4 NÁVRH VLASTNÉHO LEXIKÁLNEHO ANALYZÁTORA

Táto kapitola sa venuje návrhu lexikálneho analyzátora pre základný imperatívny jazyk. Návrh vychádza z teoretických poznatkov o regulárnych výrazoch a konečných automatoch.

Cieľom je navrhnúť analyzátor, ktorý bude schopný rozpoznať základné lexikálne jednotky vstupného textu a transformovať ich na postupnosť tokenov vhodných pre ďalšie fázy spracovania.

4.1 Špecifikácia imperatívneho jazyka

Imperatívne programovacie jazyky sú založené na postupnom vykonávaní príkazov, ktoré menia stav programu. Program je v nich chápaný ako sekvencia inštrukcií, ktoré operujú nad premennými a určujú presný postup výpočtu. Medzi základné konštrukcie imperatívnych jazykov patria priradenie hodnôt, riadiace štruktúry a bloky príkazov [3].

Navrhovaný lexikálny analyzátor je určený pre spracovanie jednoduchého imperatívneho jazyka, ktorý predstavuje zjednodušený model bežných programovacích jazykov.

Jazyk obsahuje základné konštrukcie ako:

- deklarácie premenných,
- priradenie hodnôt,
- aritmetické a relačné operácie,
- riadiace štruktúry (napr. podmienky a cykly).

Bloky kódu sú ohraničené zátvorkami a syntax je inšpirovaná jazykom C. Identifikátory môžu obsahovať písmená, číslice a podčiarkovník, pričom prvý znak musí byť písmeno alebo podčiarkovník.

Rozsah jazyka je zámerne obmedzený, aby bolo možné demonštrovať princípy lexikálnej analýzy bez zbytočnej komplexnosti. Jazyk obsahuje len tie prvky, ktoré sú

nevyhnutné pre reprezentáciu základných programových konštrukcií a zároveň umožňujú rozpoznávanie lexikálnych jednotiek.

4.1.1 Príklad vstupného programu

Nasledujúci príklad ilustruje typický vstup pre lexikálny analyzátor:

```
int fibonacci(int n) {
    int a = 0;
    int b = 1;
    int i = 0;

    while (i < n) {
        a = a + b;
        i = i + 1;
    }

    return a;
}
```

4.2 Architektúra riešenia

Navrhovaný lexikálny analyzátor je implementovaný ako modulárny systém pozostávajúci z viacerých logických komponentov. Každý komponent zabezpečuje konkrétnu fázu spracovania vstupu, čo zodpovedá bežnému návrhu kompilátorov.

Architektúra systému je rozdelená do nasledujúcich častí:

- spracovanie regulárnych výrazov,
- konštrukcia nedeterministického automatu (NFA),
- prevod na deterministický automat (DFA),
- samotná lexikálna analýza.

Toto rozdelenie umožňuje jasné oddelenie jednotlivých fáz spracovania a zjednodušuje rozšíriteľnosť a údržbu systému.

Proces spracovania vstupu prebieha nasledovne:

- Regulárne výrazy definujúce tokeny sú transformované na NFA pomocou Thompsonovej konštrukcie.
- NFA je následne prevedený na DFA pomocou konštrukcie podmnožín.
- Výsledný DFA je použitý na efektívne rozpoznávanie tokenov vo vstupnom texte.
- Lexikálny analyzátor generuje postupnosť tokenov, ktorá slúži ako vstup pre ďalšie fázy spracovania.

Takto navrhnutá architektúra reflektuje štandardný postup konštrukcie lexikálnych analyzátorov v kompilátoroch.

5 IMPLEMENTÁCIA

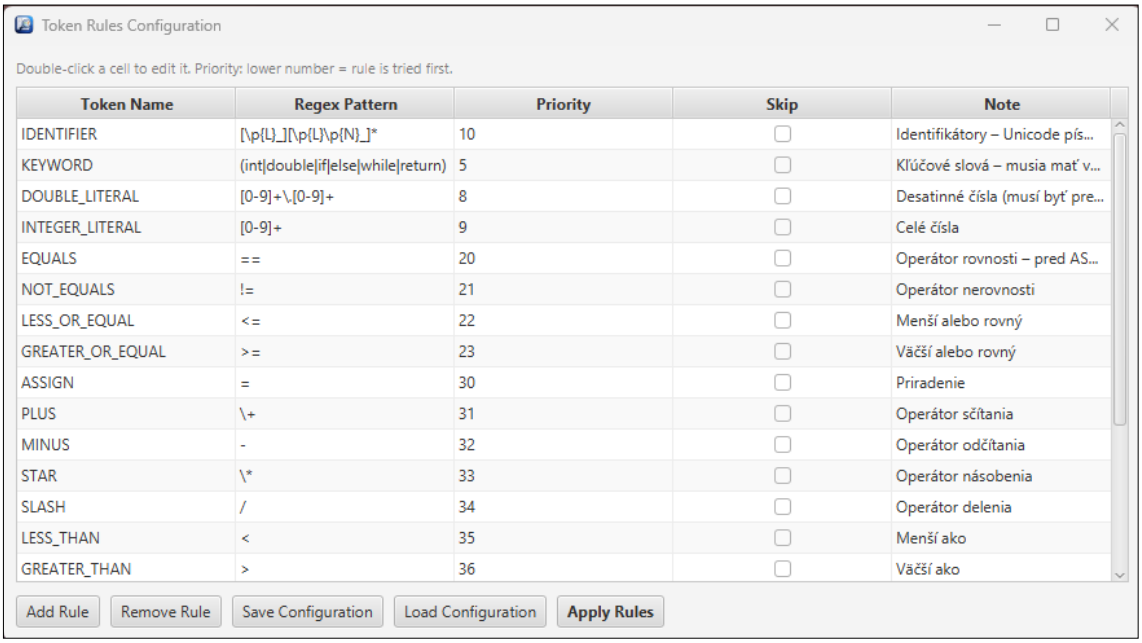
Táto kapitola skôr pojednáva o tom ako boli aplikované teoretické poznatky pri implementácii lexikálneho analyzátora avšak sa nevenuje podrobným technickým detailom.

Na rozdiel od predchádzajúcej kapitoly, ktorá popisovala návrh, sa tu pozornosť sústreďuje na praktické spracovanie regulárnych výrazov, konštrukciu automatov a samotný algoritmus lexikálnej analýzy.

5.1 Definovanie tokenizačných pravidiel

Vstupom pre spracovanie regulárnych výrazov je množina tokenizačných pravidiel, ktoré definujú akým spôsobom sa budú rozpoznávať lexémy vo vstupnom texte.

Každé pravidlo pozostáva z názvu, regulárneho výrazu, priority, možnosti preskočenia (využiteľné napríklad pri whitespaces) a poznámky pre používateľa. Regulárne výrazy sú po načítaní transformované do internej štruktúry, ktorá umožňuje ich ďalšie spracovanie. Táto štruktúra je reprezentovaná operáciami zreťazenie, alternácia a Kleeneho uzáver, ktorá je následne využitá pri konštrukcii NFA.



Token Name	Regex Pattern	Priority	Skip	Note
IDENTIFIER	[\p{L}_][\p{L}\p{N}_]*	10	☐	Identifikátory – Unicode pís...
KEYWORD	(int|double|if|else|while|return)	5	☐	Kľúčové slová – musia mať v...
DOUBLE_LITERAL	[0-9]+\.[0-9]+	8	☐	Desatinné čísla (musí byť pre...
INTEGER_LITERAL	[0-9]+	9	☐	Celé čísla
EQUALS	==	20	☐	Operátor rovnosti – pred AS...
NOT_EQUALS	!=	21	☐	Operátor nerovnosti
LESS_OR_EQUAL	<=	22	☐	Menší alebo rovný
GREATER_OR_EQUAL	>=	23	☐	Väčší alebo rovný
ASSIGN	=	30	☐	Priradenie
PLUS	\+	31	☐	Operátor sčítania
MINUS	-	32	☐	Operátor odčítania
STAR	*	33	☐	Operátor násobenia
SLASH	/	34	☐	Operátor delenia
LESS_THAN	<	35	☐	Menší ako
GREATER_THAN	>	36	☐	Väčší ako

Obrázok 5.1 – Obrazovka s definíciami tokenizačných pravidiel

Zdroj: (vlastné spracovanie)

5.2 Implementácia regulárnych výrazov

Implementácia regulárnych výrazov predstavuje prvý krok pri spracovaní tokenizačných pravidiel. V tejto fáze sa vstupný výraz zadaný používateľom nevyužíva priamo v textovej podobe, ale postupne sa transformuje na internú reprezentáciu vhodnú na ďalšie algoritmické spracovanie. Cieľom tejto časti implementácie je pripraviť regulárny výraz do takej formy, aby mohol byť následne použitý pri konštrukcii nedeterministického konečného automatu.

5.2.1 Spracovanie regulárneho výrazu

Základ implementácie tvoria triedy **RegexTokenType**, **RegexToken** a **RegexPattern**. Trieda **RegexTokenType** definuje množinu typov tokenov, ktoré sa môžu v regulárnom výraze vyskytovať. Patria sem bežné znaky, operátory alternácie, zreťazenia a kvantifikátorov, ako aj zátvorky určujúce skupiny. Trieda **RegexToken** následne reprezentuje konkrétny prvok výrazu a uchováva jeho typ, prípadne aj hodnotu v prípade obyčajného znaku. Trieda **RegexPattern** slúži ako obal celej špecifikácie regulárneho výrazu a zároveň vykonáva základnú kontrolu korektnosti vstupu, napríklad overenie, že výraz nie je prázdny a že zátvorky sú vyvážené.

Takéto riešenie umožňuje oddeliť pôvodný textový zápis výrazu od jeho spracovanej podoby. Namiesto opakovanej práce so znakmi vstupného reťazca tak ďalšie časti systému pracujú s presne definovanými tokenmi. Tým sa spracovanie výrazu stáva prehľadnejším a zároveň menej náchylným na chyby. Samotné spracovanie vstupu zabezpečuje trieda **RegexTokenizer**, ktorá prechádza znak po znaku a rozpoznáva, či ide o obyčajný znak, operátor alebo zátvorku. Výsledkom tejto fázy je lineárna postupnosť tokenov, ktorá už nepredstavuje iba text, ale formálne rozdelený výraz pripravený na ďalšie úpravy.

5.2.2 Explicitné zreťazenie

Pri zápise regulárnych výrazov sa operátor zreťazenia zvyčajne neuvádza explicitne. Ak používateľ zadá napríklad výraz ab, jeho význam je zrejmý, avšak pre implementáciu nie je vhodné ponechať túto operáciu iba implicitne. Z tohto dôvodu sa po základnej tokenizácii vykonáva ešte ďalší krok, v ktorom sa medzi vhodné susedné tokeny doplní explicitný operátor $CONCAT$.

Tento krok je dôležitý preto, že odstraňuje nejednoznačnosť pri ďalšom spracovaní výrazu. Systém už nemusí ďalej odvodzovať, kde sa zreťazenie nachádza, ale pracuje s úplnou reprezentáciou, v ktorej sú všetky operácie vyjadrené priamo. Operátor zreťazenia sa vkladá medzi také dvojice tokenov, kde ľavá časť môže prirodzene prechádzať do pravej, napríklad medzi dva znaky, medzi znak a otváraciu zátvorku alebo medzi kvantifikátor a nasledujúci znak. Týmto sa vytvorí presnejšia interná forma výrazu, ktorá je vhodná na prevod do postfixového zápisu.

Z implementačného hľadiska ide o pomerne nenápadný, ale veľmi podstatný krok. Práve doplnenie explicitného zreťazenia zjednodušuje ďalšie algoritmy, pretože všetky operácie sú následne spracovávané rovnakým spôsobom a nie je potrebné osobitne

riešiť implicitné pravidlá zápisu. Výraz sa tým dostáva do podoby, ktorá už viac zodpovedá potrebám systému než pôvodnej používateľskej notácii.

5.2.3 Postfixová forma

Po tokenizácii a doplnení explicitného zreťazenia nasleduje prevod výrazu do postfixovej formy. Tento krok zabezpečuje trieda **RegexToPostfixConverter**, ktorá využíva algoritmus *Shunting-yard*. Dôvodom prevodu je najmä to, že postfixový zápis umožňuje jednoduchšie a jednoznačnejšie spracovanie operátorov pri následnej konštrukcii automatu. Namiesto priebežného riešenia priorít a zátvoriek sa operátory v postfixovej forme spracúvajú až po svojich operandoch, čo prirodzene vyhovuje zásobníkovému spôsobu konštrukcie.

V implementácii sú jednotlivým operátorom priradené priority. Najvyššiu prioritu majú unárne operátory ako $*$, $+$ a ?, nasleduje zreťazenie a najnižšiu prioritu má alternácia. Pri spracovaní tokenov sa znaky odosielajú priamo na výstup, zatiaľ čo operátory sa ukladajú na zásobník alebo sa z neho podľa potreby presúvajú na výstup. Zátvorky pritom určujú hranice lokálneho spracovania podobne ako pri bežnom vyhodnocovaní výrazov. Výsledkom je postfixový zoznam tokenov, ktorý už zachytáva správne poradie operácií bez potreby dodatočnej interpretácie.

Takto pripravený výraz tvorí priamy vstup pre konštrukciu NFA. Výhodou postfixovej formy je, že jednotlivé operácie možno pri budovaní automatu aplikovať postupne nad zásobníkom čiastkových fragmentov. Prechod z regulárneho výrazu do tejto podoby teda nepredstavuje iba formálnu úpravu zápisu, ale dôležitý implementačný medzikrok, ktorý prepája textovú definíciu tokenu s automatovou reprezentáciou použitou v ďalšej časti systému.

5.3 Implementácia NFA

Po spracovaní regulárneho výrazu do postfixovej formy nasleduje vytvorenie NFA. Táto fáza predstavuje prvý krok, v ktorom sa formálna špecifikácia tokenu premieňa na vykonateľnú automatovú štruktúru. V navrhnutom riešení je NFA zostavovaný postupne z menších častí podľa jednotlivých tokenov regulárneho výrazu, pričom samotnú konštrukciu zabezpečuje trieda ThompsonConstructor. Výsledkom tejto fázy nie je ešte automat vhodný na priame rozpoznávanie vstupu, ale „medzireprezentácia“, ktorá sa neskôr využíva pri konštrukcii deterministického automatu.

5.3.1 Reprezentácia NFA

Nedeterministický konečný automat je v implementácii reprezentovaný trojicou tried **NFAState**, **NFATransition** a **NFA**. Každý stav automatu je reprezentovaný objektom **NFAState**, ktorý uchováva svoje identifikačné číslo, príznak akceptácie a zoznam odchádzajúcich prechodov. Samotný prechod je zapuzdrený v triede **NFATransition**, kde sa eviduje zdrojový stav, cieľový stav a vstupný symbol. Osobitné postavenie má hodnota *null*, ktorá označuje ε-prechod (*kapitola 2, časť 2.2*), teda prechod uskutočnený bez čítania symbolu zo vstupu. Práve ε-prechody sú dôležité pri spájaní menších častí automatu do väčších celkov.

Trieda **NFA** reprezentuje celý automat ako jeden celok. Uchováva odkaz na počiatočný stav, zoznam všetkých stavov a samostatný zoznam akceptačných stavov. Takéto riešenie je praktické najmä z pohľadu ďalších fáz spracovania, pretože akceptačné stavy nie je potrebné opakovane vyhľadávať v celej štruktúre. Popri tom sa pri konštrukcii využíva aj pomocný objekt **NFAFragment**, ktorý reprezentuje neúplný automat s jedným počiatočným a jedným koncovým stavom. Namiesto vytvárania celého automatu naraz sa teda systém opiera o skladanie menších fragmentov, čo robí implementáciu prehľadnejšou a lepšie zodpovedajúcou logike samotného regulárneho výrazu.

5.3.2 Konštrukcia NFA

Samotná výstavba automatu prebieha v triede **ThompsonConstructor**, ktorá spracúva tokeny regulárneho výrazu v postfixovej forme. Pri spracovaní sa využíva zásobník objektov typu **NFAFragment**. Každý token vyvolá konkrétnu konštrukčnú operáciu, ktorá buď vytvorí nový fragment, alebo spojí už existujúce fragmenty do väčšieho celku. Takýto postup je z implementačného hľadiska výhodný, pretože prirodzene rešpektuje štruktúru postfixového zápisu a zároveň umožňuje budovať automat krok po kroku bez potreby spätnej analýzy priorít operátorov.

Pri tokene *CHARACTER* sa vytvorí najjednoduchší fragment pozostávajúci z dvoch stavov a jedného prechodu medzi nimi. Pri operátore $CONCAT$ sa spoja dva fragmenty tak, že koncový stav prvého sa prepojí ε-prechodom so začiatočným stavom druhého. Pri alternácii vzniká nový počiatočný aj nový koncový stav, medzi ktoré sú vložené obe vetvy výrazu. Operátory $KLEENE_STAR$, $PLUS$ a $OPTIONAL$ vytvárajú nové prepojenia tak, aby podporovali opakovanie, aspoň jeden výskyt alebo voliteľný výskyt danej časti výrazu. Dôležité je, že pri každom z týchto krokov sa existujúce fragmenty nemenia svojou vnútornou logikou, ale rozširujú sa o nové stavy a ε-prechody. Tým sa zachováva jednoduchosť celej konštrukcie a zároveň sa znižuje riziko chýb pri spracovaní zložitejších regulárnych výrazov.

5.3.3 Pripravenosť na prevod do DFA

Po spracovaní všetkých tokenov zostane na zásobníku jediný fragment, ktorý predstavuje výsledný NFA pre celý regulárny výraz. Koncový stav tohto fragmentu sa označí ako akceptačný a následne sa z počiatočného stavu prehľadávaním zozbierajú všetky dosiahnuteľné stavy. Tie sa následne zaregistrujú do objektu NFA, čím vznikne kompletná reprezentácia automatu pripravená na ďalšie spracovanie. Tento krok je dôležitý preto, aby výsledný objekt neobsahoval iba lokálne vytvorené fragmenty, ale uzavretú štruktúru predstavujúcu celý automat.

Takto vytvorený NFA ešte nie je určený na finálne rozpoznávanie lexém vo vstupnom texte. Jeho hlavnou úlohou je slúžiť ako prechodná reprezentácia medzi regulárnym výrazom a deterministickým automatom. Výhodou tohto prístupu je, že konštrukcia NFA je priamočiara, dobre zodpovedá štruktúre regulárneho výrazu a dá sa jednoducho implementovať pomocou skladania fragmentov. Zároveň tým vzniká pevný základ pre nasledujúcu fázu, v ktorej sa nedeterministický automat prevádza na deterministický pomocou konštrukcie podmnožín.

5.4 Implementácia DFA

Po vytvorení NFA nasleduje jeho prevod na DFA. Tento krok je v implementácii dôležitý najmä preto, že samotné rozpoznávanie vstupného textu je pri deterministickom automate jednoduchšie a efektívnejšie. Zatiaľ čo pri NFA môže z jedného stavu pre rovnaký symbol viesť viacero možností, DFA zabezpečuje, že pre každý symbol existuje z aktuálneho stavu najviac jeden ďalší stav. V navrhnutom riešení je táto transformácia realizovaná triedou **SubsetConstruction**, ktorá vytvára DFA na základe už zostaveného NFA.

5.4.1 Reprezentácia DFA

Deterministický konečný automat je reprezentovaný triedami **DFAState** a **DFA**. Trieda **DFAState** uchováva identifikačné číslo stavu, príznak akceptácie a prechody vo forme mapy **Map<Character, DFAState>**. Takéto riešenie priamo vychádza z vlastnosti deterministického automatu, podľa ktorej môže pre každý vstupný symbol existovať najviac jeden nasledovný stav. Na rozdiel od NFA teda nie je potrebné evidovať zoznam viacerých prechodov pre ten istý symbol, ale postačuje jednoznačné mapovanie symbolu na cieľový stav.

Okrem toho každý stav DFA uchováva aj množinu stavov NFA, z ktorých vznikol. Táto informácia má zásadný význam pri samotnej konštrukcii, pretože jeden stav DFA v skutočnosti reprezentuje celú podmnožinu stavov pôvodného nedeterministického automatu. Trieda **DFA** následne združuje počiatočný stav, zoznam všetkých stavov a zoznam akceptačných stavov. Podobne ako pri NFA ide o praktické riešenie, ktoré zjednodušuje ďalšiu prácu s automatom a umožňuje rýchle zisťovanie, či sa automat nachádza v prijímajúcom stave.

5.4.2 Konštrukcia DFA

Samotná výstavba deterministického automatu je realizovaná algoritmom konštrukcie podmnožín (*kapitola 2, časť 2.5*). Tento algoritmus vychádza z myšlienky, že jeden stav DFA reprezentuje všetky stavy NFA, v ktorých by sa automat mohol nachádzať po spracovaní určitej časti vstupu.

Trieda **SubsetConstruction** následne pracuje s pracovným zoznamom stavov DFA, ktoré ešte neboli úplne spracované. Pre každý takýto stav a pre každý symbol abecedy sa najprv určí množina stavov NFA dosiahnuteľných po prečítaní daného symbolu. Ak takáto množina ešte nebola vytvorená ako stav DFA, vytvorí sa nový stav a zaradí sa do pracovného zoznamu. V opačnom prípade sa použije už existujúci stav. Týmto spôsobom sa postupne zostaví celý deterministický automat bez toho, aby vznikali duplicitné stavy.

Praktickým detailom implementácie je spôsob identifikácie jednotlivých podmnožín stavov NFA. Na tento účel sa používa textový kľúč vytvorený zoradením identifikátorov stavov a ich spojením do jedného reťazca. Takéto riešenie zabezpečuje, že rovnaká množina stavov bude vždy reprezentovaná rovnakým kľúčom, bez ohľadu na poradie, v akom sa prvky nachádzali v množine. Vďaka tomu možno jednoducho overiť, či už bol daný stav DFA vytvorený, a predísť opakovanému generovaniu tej istej podmnožiny.

5.4.3 Výsledná podoba DFA

Po ukončení algoritmu vznikne úplný deterministický konečný automat, ktorého stavy a prechody už umožňujú jednoznačné spracovanie vstupného textu. Stav DFA je označený ako akceptačný vtedy, ak jeho podmnožina obsahuje aspoň jeden akceptačný stav NFA. Takýto prístup priamo zachováva význam pôvodného nedeterministického automatu, no zároveň prevádza jeho správanie do formy, ktorá je vhodnejšia na praktické použitie.

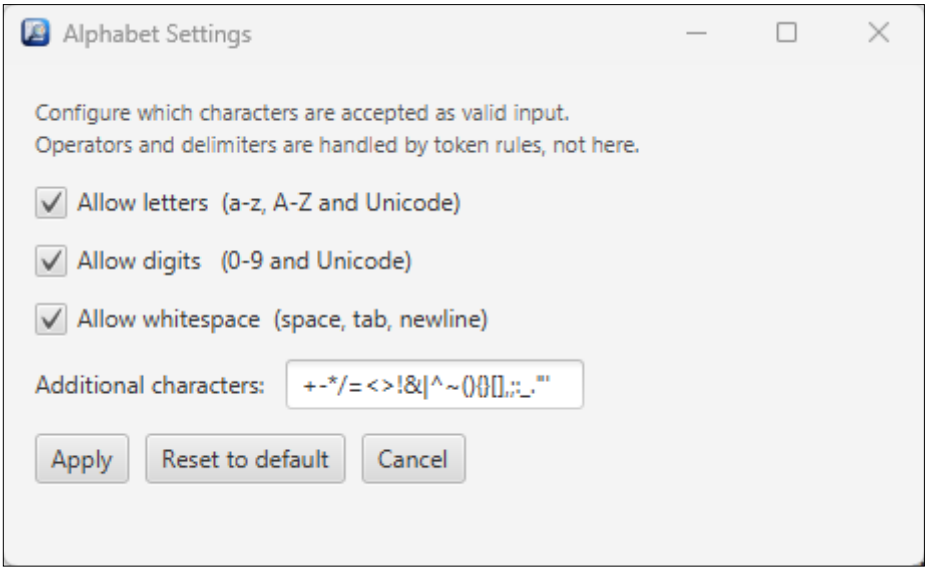
Z pohľadu lexikálnej analýzy má DFA zásadný význam, pretože práve na jeho základe možno nad vstupným reťazcom vykonávať efektívne rozpoznávanie lexém. Pri spracovaní každého znaku nie je potrebné sledovať viacero možných vetiev výpočtu, ale automat sa vždy presúva do jedného presne určeného stavu. Tým sa zjednodušuje samotná implementácia analyzátora a zároveň sa zlepšuje jeho výkonnosť. Výsledný DFA tak predstavuje finálnu automatovú reprezentáciu tokenizačných pravidiel a uzatvára reťazec spracovania, ktorý sa začal pri textovej definícii regulárneho výrazu.

5.5 Implementácia lexera

Lexikálny analyzátor je implementovaný ako komponent, ktorý pracuje nad DFA a spracováva vstupný text po jednotlivých znakoch.

5.5.1 Konfigurácia abecedy

Pred spracovaním vstupného textu je potrebné definovať množinu znakov, ktoré môže lexikálny analyzátor spracovávať. Táto množina predstavuje vstupnú abecedu analyzátora a je konfigurovateľná pomocou grafického rozhrania.



Obrázok 5.2 – Konfiguračné okno vstupnej abecedy

Zdroj: (vlastné spracovanie)

5.5.2 Princíp najdlhšej zhody

Často pri spracovávaní vstupného textu nastáva situácia, keď viacero tokenizačných pravidiel zodpovedá rovnakému prefixu vstupu. Na riešenie takejto situácie v implementácii sa využíva princíp najdlhšej zhody.

Podstata je v tom, že lexikálny analyzátor sa snaží rozpoznať čo najdlhší úsek vstupu, ktorý zodpovedá niektorému z definovaných tokenov. Inak povedané, analyzátor pokračuje v čítaní znakov dovtedy, kým exituje platný prechod v automate, pričom si pamätá posledný dosiahnutý akceptačný stav.

Príkladom situácie kedy je aplikovaný princíp najdlhšej zhody môže byť rozlišovanie medzi operátormi rôznej dĺžky (napr. = a ==).

V implementovanom lexikálnom analyzátore je ošetrená aj situácia kedy princíp najdlhšej zhody nestačí ak dve rôzne tokenizačné pravidlá nájdu zhodu rovnakej dĺžky – vtedy rozhodne priorita, ktorá je zadaná manuálne *Obrázok 5.1*.

Príkladom takejto situácie môže byť keď pravidlá KEYWORD s prioritou 1 a IDENTIFIER s prioritou 2 nájdu zhodu rovnakej dĺžky napríklad pri lexéme „int“. Bez priorít by sa mohlo stať, že analyzátor nebude vedieť priradiť k tejto lexéme správny token. V tomto prípade bude lexéma „int“ priradená tokenu KEYWORD keďže ten má vyššiu prioritu.

5.5.3 Algoritmus spracovania vstupu

Samotné spracovanie vstupu prebieha sekvenčne pomocou deterministického konečného automatu.

Analyzátor postupuje nasledovne:

1. nastaví sa počiatočný stav automatu,
2. postupne sa čítajú znaky vstupu,
3. pre každý znak sa vykoná prechod do nasledujúceho stavu,
4. priebežne sa zaznamenáva posledný dosiahnutý akceptačný stav spolu s jeho pozíciou vo vstupnom texte.

V prípade, že pre aktuálny znak neexistuje platný prechod:

1. analyzátor sa vráti na pozíciu posledného akceptačného stavu,
2. vytvorí token z najdlhšieho rozpoznaného lexému,
3. spracovanie pokračuje od nasledujúcej pozície za touto lexémou opätovne od počiatočného stavu.

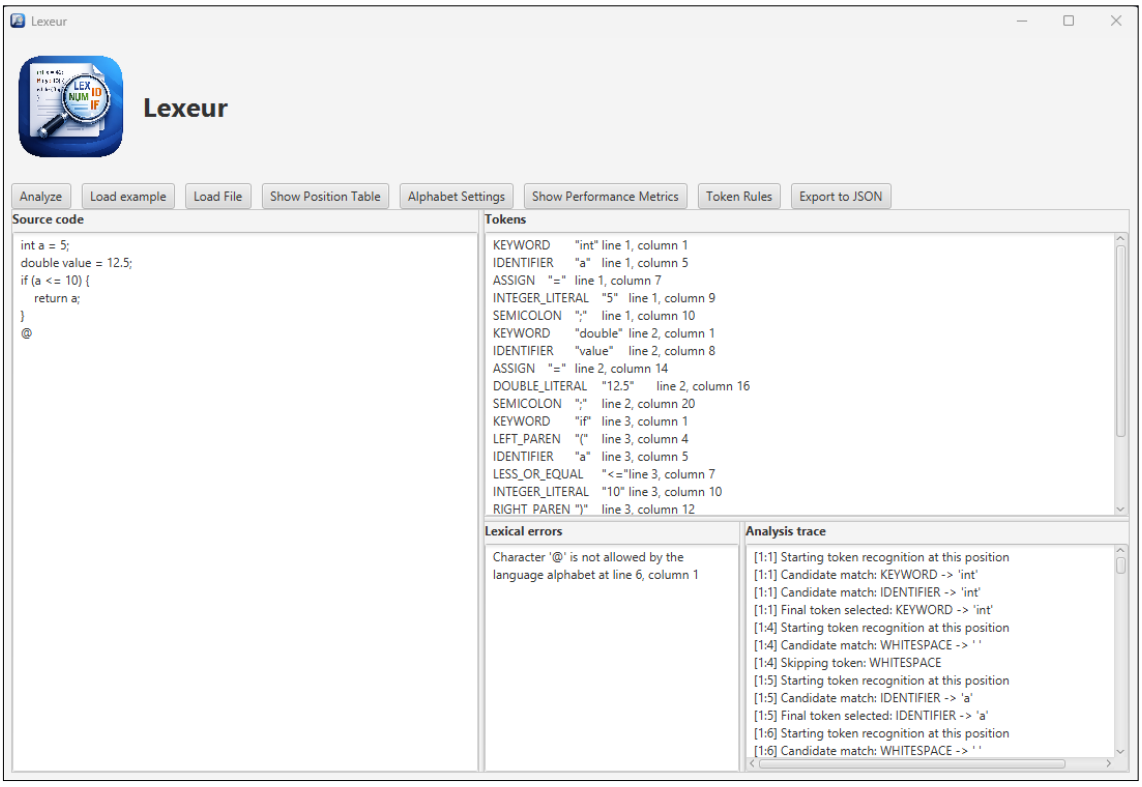
Tento postup implementuje princíp najdlhšej zhody, ktorý zabezpečuje správne rozpoznávanie tokenov aj v prípade, že viacero pravidiel zodpovedá rovnakému prefixu vstupu.

Špeciálnym prípadom sú znaky, ktoré nemajú význam pre lexikálnu analýzu (napr. whitespace). Tieto znaky sú počas spracovania ignorované a nevytvárajú samostatné tokeny. Naopak, pri výskyte znaku, pre ktorý neexistuje žiadny prechod a zároveň nebol dosiahnutý žiadny akceptačný stav, je zaznamenaná lexikálna chyba,

5.6 Generovanie tokenov

Výsledkom lexikálnej analýzy je postupnosť tokenov. Pri dosiahnutí akceptačného stavu identifikovaná lexéma priradí ku konkrétnemu typu tokenu. V prípade výskytu lexikálnej chyba, pričom analýza pokračuje až do konca.

Výstup analyzátora je možné ďalej spracovať napríklad v syntaktickej analýze alebo exportovať do štruktúrovanej podoby napríklad do formátu JSON.



Obrázok 5.3 – Výpis tokenov a lexikálnych chýb

Zdroj: (vlastné spracovanie)

6 TESTOVANIE A VYHODNOTENIE

Testovanie má za cieľ overiť správnosť rozpoznávania tokenov, spracovania chybových stavov a základných výkonnostných charakteristík.

6.1 Testovacie scenáre

Testovanie lexikálneho analyzátora bolo realizované na viacerých scenároch, ktoré pokrývajú štandardné aj menej štandardné použitie analyzátora. Cieľom bolo overiť nielen správnosť spracovania vstupu, ale aj flexibilitu navrhnutého riešenia.

6.1.1 Imperatívny jazyk

Základný testovací scenár predstavuje konfiguráciu tokenov pre jednoduchý imperatívny jazyk, ktorý obsahuje základné konštrukcie ako deklarácie premenných, podmienky a návratové príkazy.

Ako prvý je uvedený jednoduchý vstup obsahujúci deklaráciu premennej a priradenie hodnoty:

```
int a = 5;
```

Takýto vstup overuje správne rozpoznanie kľúčového slova, identifikátora, operátora priradenia, číselného literálu, ako aj spracovanie ukončovacieho znaku.

Nasledujúci vstup reprezentuje podmienkový príkaz:

```
if (a <= 10) {
    return a;
}
```

V tomto prípade sa overuje správne spracovanie riadiacej štruktúry, zátvoriek a blokov kódu, zároveň aj aplikácia princípu najdlhšej zhody pri rozpoznávaní porovnávacieho operátora.

Posledný vstup obsahuje deklaráciu premennej s desatinou hodnotou:

```
double hodnota = 12.5;
```

Tento vstup je zameraný na overenie správneho rozpoznania desatinného číselného literálu.

6.1.2 Aritmetické výrazy

Scenár je zameraný na spracovanie aritmetických výrazov. Použitá konfigurácia obsahuje minimálnu množinu tokenov potrebnú na vyjadrenie matematických operácií.

V tomto prípade bol použitý výraz obsahujúci viacero operácií a zátvorky:

```
3.14 * (2 + 5) / 7
```

Príklad vstupu overuje správne rozpoznanie aritmetických operátorov a zátvoriek.

6.1.3 Formát CSV

Tretí scenár demonštruje použitie lexikálneho analyzátora mimo oblasti programovacích jazykov. Konfigurácia tokenov je prispôsobená formátu CSV (Comma-Separated Values).

Cieľom scenára je overiť, či je analyzátor schopný správne spracovať jednoduchý textový formát pozostávajúci z hodnôt oddelených čiarkami, pričom niektoré hodnoty môžu byť uzavreté v úvodzovkách.

```
value1,value2,"value, with comma",42
```

6.1.4 Chybné vstupy

Posledný scenár je zameraný na testovanie robustnosti analyzátora a spracovanie chybových stavov.

Ako prvý bol použitý vstup obsahujúci neznámy znak:

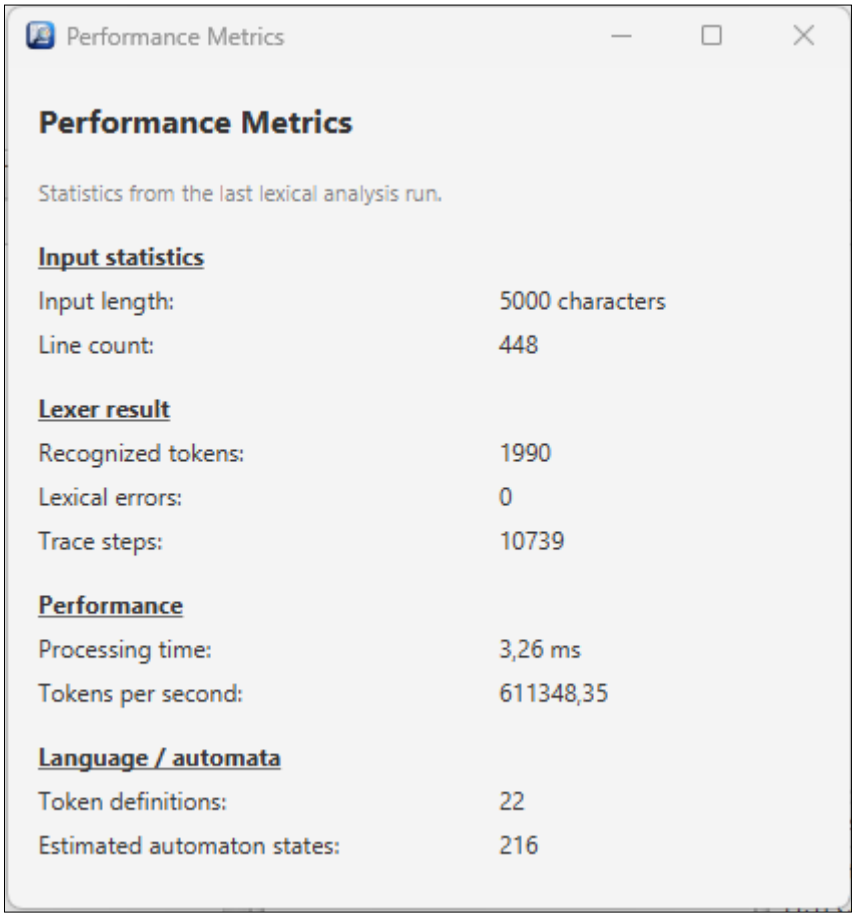
```
@
```

Druhý vstup obsahuje neukončený reťazec:

```
"neukonceny retazec
```

6.2 Výkonnostné testy

Výkon analyzátora bol meraný na vstupoch rôznej dĺžky pomocou zabudovaného výkonnostného meradla v analyzátore. Cieľom bolo overiť ako sa mení čas spracovania v závislosti od veľkosti vstupu.



Obrázok 6.1 – Okno s prehľadom výkonnostných metrík

Zdroj: (vlastné spracovanie)

6.2.1 Experiment

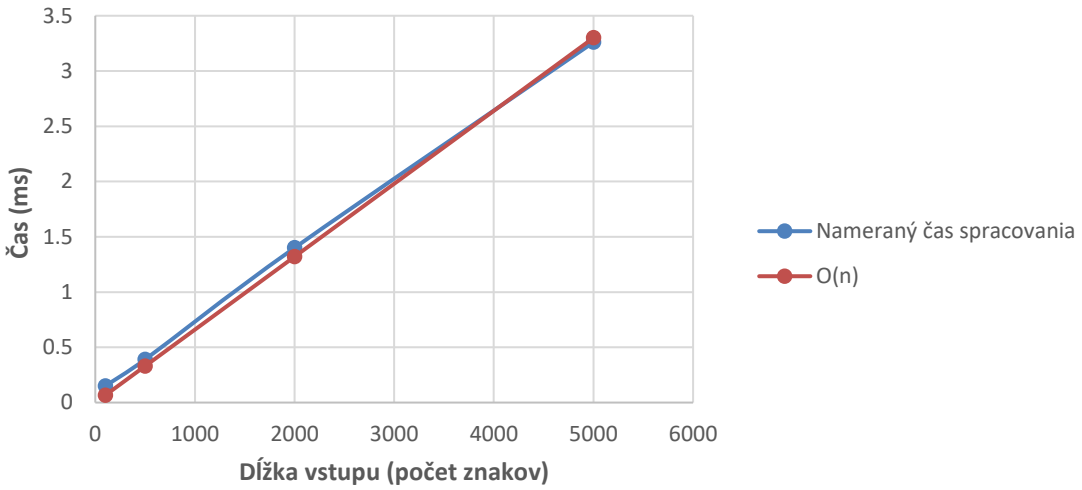
Tabuľka 6.1 – Meranie času spracovania vstupu

Dĺžka vstupu (znaky)	Počet tokenov	Čas (ms)
~100	40	0,15
~500	200	0,39
~2000	800	1,40
~5000	2000	3,26

Namerané hodnoty času spracovania môžu byť skreslené vplyvom JVM a jej JIT kompilátora.

Na účely vyhodnotenia výkonnosti lexikálneho analyzátora boli vykonané experimenty na vstupoch rôznej dĺžky. Testovacie vstupy obsahovali približne 100, 500, 2000 a 5000 znakov. Pre každý vstup bol zaznamenaný čas spracovania v milisekundách.

Získané hodnoty sú graficky znázornené v podobe závislosti času spracovania od dĺžky vstupu. V grafe je okrem nameraných údajov doplnená aj lineárna funkcia $O(n)$, ktorá reprezentuje očakávaný priebeh časovej zložitosti.



Obrázok 6.2 – Graf závislosti času od spracovania dĺžky vstupu

Zdroj: (vlastné spracovanie)

Z grafu je zrejmé, že čas spracovania rastie približne lineárne s dĺžkou vstupu. Namerané hodnoty sa zhodujú s lineárnou funkciou $O(n)$, čo naznačuje, že implementovaný lexikálny analyzátor spracováva vstup efektívne v lineárnom čase.

6.3 Porovnanie s Flex a ANTLR

Kým Flex aj ANTLR pracujú na princípe generovania zdrojového kódu analyzátora pred jeho spustením, navrhnuté riešenie zvolilo odlišný prístup – lexikálnu analýzu vykonáva priamo za behu na základe dynamicky definovaných pravidiel, tak ako bolo znázornené v kapitole 5. Dôsledkom toho odpadá potreba generovania a rekompilácie pri každej zmene konfigurácie tokenov. Nové pravidlá možno pridávať, upravovať a okamžite testovať priamo v grafickom rozhraní aplikácie.

Spoločnou črtou všetkých troch riešení je uplatnenie princípu najdlhšej zhody pri rozpoznávaní tokenov. Navrhnutý analyzátor však na rozdiel od Flexu a ANTLR pravidlá nevyhodnocuje prostredníctvom predgenerovaného deterministického konečného automatu, ale sekvenčne aplikuje regulárne výrazy zoradené podľa priorít, čím sa líši aj mechanizmus riešenia konfliktov medzi pravidlami.

Oba porovnávané nástroje sú primárne určené na generovanie kódu pre konkrétny cieľový jazyk – Flex pre C/C++, ANTLR pre viacero jazykov vrátane Javy či Pythonu. Navrhnuté riešenie nie je viazané na žiadny cieľový jazyk ani formát; analyzátor možno použiť aj na spracovanie iných textových formátov, ako napríklad CSV alebo JSON, bez akéhokoľvek zásahu do implementácie.

Na rozdiel od oboch nástrojov navrhnuté riešenie poskytuje grafické rozhranie s podrobným zaznamenávaním krokov analýzy, tabuľkou pozícií tokenov a možnosťou

exportu výsledkov do formátu JSON. Tieto vlastnosti robia z aplikácie nástroj vhodný predovšetkým na interaktívne experimentovanie s konfiguráciou tokenizácie.

Výkon analyzátora je nižší v porovnaní s nástrojmi Flex a ANTLR, keďže pravidlá sú vyhodnocované interpretovane pri každom spustení analýzy, zatiaľ čo nástroje Flex a ANTLR generujú optimalizovaný tabuľkovo riadený deterministický konečný automat vopred – pred samotným spustením analyzátora. Toto obmedzenie je akceptovateľné pre kratšie vstupy typické pri interaktívnom použití, avšak pri spracovaní rozsiahlych zdrojových súborov by bol rozdiel vo výkone výraznejší.

Druhým obmedzením je rozsah analýzy – kým ANTLR pokrýva lexikálnu aj syntaktickú rovinu a automaticky konštruuje syntaktický strom, navrhnuté riešenie sa zameriava výlučne na lexikálnu analýzu. Pre účely tejto práce je však toto vymedzenie zámerné, keďže implementácia syntaktickej analýzy by výrazne presahovala stanovený rozsah práce.

7 DISKUSIA VÝSLEDKOV

Na základe vykonaných testov možno konštatovať, že implementovaný lexikálny analyzátor sa vo všetkých testovacích scenároch správal v súlade s očakávaniami. Zvolené vstupy pokrývali základné aj menej štandardné prípady použitia, čím bolo možné overiť nielen správnosť spracovania, ale aj flexibilitu riešenia.

V rámci scenára imperatívneho jazyka bolo overené správne rozpoznávanie základných jazykových konštrukcií, ako sú kľúčové slová, identifikátory, operátory a číselné literály. Analyzátor konzistentne aplikoval princíp najdlhšej zhody, čo sa prejavilo napríklad pri spracovaní viacznakových operátorov. Správne fungovalo aj rozlíšenie medzi identifikátormi a rezervovanými slovami.

Scenár aritmetických výrazov potvrdil schopnosť analyzátora spracovať výrazy obsahujúce rôzne typy operátorov a číselných literálov. V tomto prípade sa ukázalo, že analyzátor je schopný pracovať aj s vlastnými tokenmi definovanými používateľom, bez nutnosti zásahu do implementácie.

Významným výsledkom je overenie flexibility riešenia prostredníctvom scenára s CSV formátom. Testy ukázali, že analyzátor je možné prispôsobiť aj na spracovanie neprogramových vstupov, pričom správne rozpoznáva hodnoty oddelené čiarkami aj reťazce uzavreté v úvodzovkách. Táto vlastnosť potvrdzuje všeobecnejší charakter navrhnutého riešenia.

Pri spracovaní chybných vstupov analyzátor preukázal dostatočnú robustnosť. Neplatné znaky alebo neúplné lexémy boli detegované a zaznamenané, pričom spracovanie vstupu pokračovalo aj po výskyte chyby. Tento prístup umožňuje získať prehľad o viacerých chybách v jednom priebehu analýzy.

Na základe vykonaného testovania možno konštatovať, že navrhnuté riešenie spĺňa stanovené ciele práce a poskytuje funkčný a flexibilný nástroj na lexikálnu analýzu textových vstupov.

8 MOŽNOSTI VYUŽITIA A ROZŠÍRENIA

Implementovaný lexikálny analyzátor bol navrhnutý s dôrazom na zrozumiteľnosť, flexibilitu a modulárnu architektúru. Tieto vlastnosti ho predurčujú nielen na edukačné účely, pre ktoré primárne vznikol, ale aj na praktické využitie v rôznych oblastiach spracovania textu. Nasledujúca kapitola predstavuje možné scenáre využitia analyzátora.

8.1 Využitie pri tvorbe vlastného programovacieho jazyka

Lexikálna analýza predstavuje prvú fázu procesu prekladu programovacieho jazyka. Výstup lexera – postupnosť tokenov – slúži ako vstup pre syntaktický analyzátor (parser), ktorý na základe gramatiky overuje správnosť štruktúry programu a vytvára abstraktný syntaktický strom (AST).

Implementovaný analyzátor preto predstavuje vhodný základ pre návrh a implementáciu jednoduchého programovacieho jazyka. Rozšírenie by zahŕňalo doplnenie definícií tokenov o prvky špecifické pre daný jazyk, ako sú napríklad reťazcové literály, komentáre alebo špecifické operátory. Následne by bolo možné nad výstupom lexera implementovať parser, napríklad metódou rekurzívneho zostupu, ktorý zabezpečí syntaktickú analýzu vstupu.

Výsledný syntaktický strom by mohol byť ďalej spracovaný napríklad interpreterom. Vďaka rozšíriteľnej architektúre, ktorá oddeľuje definície tokenov, samotnú lexikálnu analýzu a používateľské rozhranie, je možné jednotlivé komponenty rozširovať nezávisle od seba.

Implementovaný analyzátor tak predstavuje vhodný nástroj pre experimentovanie s návrhom jazykov

8.2 Využitie pri spracovaní doménovo špecifických jazykov

Doménovo špecifické jazyky (DSL) sú navrhnuté pre konkrétnu aplikačnú oblasť a vyznačujú sa jednoduchšou štruktúrou než všeobecné programovacie jazyky. Práve táto jednoduchosť robí z implementovaného analyzátora vhodný základ pre ich spracovanie.

Ukážka takéhoto DSL môže vyzerať nasledovne:

```
On button: 1 -> MPS
Activity: Description on button: 1 Playback T
      Get info
```

Lexikálny analyzátor by pri spracovaní takéhoto vstupu identifikoval napríklad kľúčové slová **On**, **Activity**, identifikátory **button**, číselné hodnoty a operátory. Výsledkom by bola štruktúrovaná postupnosť tokenov, ktorá by mohla byť následne spracovaná interpretačným modulom.

Takýto prístup umožňuje vytvárať jednoduché, prehľadné a ľahko upraviteľné jazyky prispôsobené konkrétnym potrebám. Implementovaný analyzátor v tomto kontexte slúži ako flexibilný nástroj pre ich rýchle prototypovanie a testovanie.

8.3 Využitie mimo kompilátorov

Lexikálna analýza nachádza uplatnenie aj mimo oblasti kompilátorov. Všade tam, kde je potrebné rozdeliť vstupný text na štruktúrované a typizované časti, môže byť lexer efektívnym nástrojom.

Jednou z oblastí je validácia vstupov, kde sú prípustné hodnoty definované pomocou formálnych pravidiel. Na rozdiel od jednoduchého použitia regulárnych výrazov umožňuje lexer rozpoznať jednotlivé komponenty vstupu a priradiť im konkrétne typy tokenov, čím poskytuje bohatšiu štruktúru výstupu.

Ďalším príkladom je spracovanie log súborov, kde majú jednotlivé riadky pevne danú štruktúru, napríklad časovú pečiatku, úroveň závažnosti a text správy. Definovaním vhodných tokenov je možné tieto časti automaticky extrahovať a ďalej spracovať.

Podobne možno analyzátor využiť pri spracovaní dátových formátov, ako sú CSV súbory alebo exportné formáty, kde bežné knižnice neposkytujú dostatočnú flexibilitu.

Implementovaný lexer tak nachádza uplatnenie v širšej oblasti spracovania textových dát, nielen pri tvorbe programovacích jazykov.

8.4 Možnosti rozšírenia

8.4.1 Paralelizácia spracovania

Pre veľmi rozsiahle vstupy by bolo možné rozdeliť vstupný reťazec na menšie úseky a spracovávať ich paralelne v samostatných vláknach.

Paralelizácia však prináša komplikácie pri presahu tokenu cez hranicu medzi dvoma úsekmi vstupu. V takýchto situáciách je potrebné zabezpečiť dodatočné spracovanie, napríklad prekrytie segmentov alebo synchronizáciu medzi vláknami.

Napriek týmto obmedzeniam môže paralelizácia predstavovať efektívnu optimalizáciu najmä pri rozsiahlych vstupoch.

ZÁVER

V rámci bakalárskej práce bol navrhnutý a implementovaný lexikálny analyzátor pre jednoduchý imperatívny jazyk, pričom sa vychádzalo z teoretických princípov regulárnych výrazov a konečných automatov. Výsledkom je aplikácia, ktorá umožňuje dynamicky definovať tokenizačné pravidlá a vykonávať lexikálnu analýzu vstupného textu v reálnom čase. Navrhnuté riešenie poskytuje flexibilný prístup k spracovaniu vstupu, keďže zmeny v definícii tokenov sa prejavia okamžite, bez potreby opätovného generovania alebo kompilácie analyzátora.

V teoretickej časti práce boli spracované základné pojmy z oblasti formálnych jazykov, regulárnych výrazov a konečných automatov. Pozornosť bola venovaná aj rozdielu medzi deterministickými a nedeterministickými konečnými automatmi, ako aj ich využitiu pri konštrukcii lexikálnych analyzátorov. Tieto poznatky predstavujú základ moderných kompilátorov, ktorých jednotlivé fázy tvoria dôležitú súčasť procesu prekladu programov.

V praktickej časti bol navrhnutý vlastný lexikálny analyzátor, ktorý sa od tradičných nástrojov, ako sú generátory typu Flex alebo ANTLR, odlišuje najmä dynamickým prístupom k definícii tokenov. Používateľ môže meniť pravidlá bez potreby generovania a rekompilácie analyzátora.

Výsledkom práce je funkčný nástroj s grafickým používateľským rozhraním, ktorý umožňuje nielen samotnú lexikálnu analýzu, ale aj kontrolu nad samotnou lexikálnou analýzou a jej ladenie. Zároveň však implementované riešenie poskytuje aj základ pre praktické využitie, napríklad pri spracovaní doménovo špecifických jazykov alebo ako súčasť jednoduchých prekladačov.

Z hľadiska osobného prínosu práca umožnila prehĺbiť znalosti z oblasti teórie formálnych jazykov a praktickej implementácie kompilátorových nástrojov. Získané skúsenosti sú využiteľné nielen v oblasti tvorby programovacích jazykov, ale aj pri spracovaní textových dát, analýze vstupov či návrhu softvérových nástrojov.

Na záver je možné konštatovať, že lexikálna analýza predstavuje základný, no zároveň kľúčový krok v procese spracovania programov.

Zoznam použitej literatúry

[1] HOPCROFT, John E. – MOTWANI, Rajeev – ULLMAN, Jeffrey D. 2001. ***Introduction to Automata Theory, Languages, and Computation***. 2. vyd. Boston: Addison-Wesley, 2001. 513 s. ISBN 0-201-44124-1.

[2] AHO, Alfred V. et al. 2007. ***Compilers: Principles, Techniques, and Tools***. 2. vyd. Boston: Pearson, 2007. 993 s. ISBN 978-0-321-48681-3.

[3] COOPER, Keith D. – TORCZON, Linda. 2012. ***Engineering a Compiler***. 2. vyd. Burlington: Elsevier, 2012. 787 s. ISBN 978-0-12-088478-0.

[4] APPEL, Andrew W. – PALSBERG, Jens. 2002. ***Modern Compiler Implementation in Java***. 2. vyd. Cambridge: Cambridge University Press, 2002. 570 s. ISBN 0-521-82060-X.

[5] THE FLEX PROJECT***. Flex: The Fast Lexical Analyzer Generator – Manual*** [online]. [cit. 10.4.2026]. Dostupné na: https://westes.github.io/flex/manual/

[6] PARR, Terence. ***ANTLR 4 Documentation*** [online]. [cit. 10.4.2026]. Dostupné na: https://www.antlr.org/

Zoznam použitých promptov

„Skontroluj logickú následnosť podkapitol v kapitole 1“, ChatGPT, verzia GPT-5.3, OpenAI, citované 18.2.2026 , chat.openai.com

„Skontroluj štylistiku a logickú následnosť pojmov v časti 1.1“, ChatGPT, verzia GPT-5.3, OpenAI, citované 21.2.2026, chat.openai.com

„Skontroluj logickú následnoť podkapitol v kapitole 2“, ChatGPT, verzia GPT-5.3, OpenAI, citované 25.2.2026, chat.openai.com

„Skontroluj správnosť príkladov v častiach 2.2 a 2.3“, ChatGPT, verzia GPT-5.3, OpenAI, citované 1.3.2026, chat.openai.com

„Vyber z kapitoly 1 a 2 zoznam symbolov a zoznam skratiek používaných v týchto kaptiloách“, “, ChatGPT, verzia GPT-5.3, OpenAI, citované 6.3.2026, chat.openai.com

„Na základe priložených dokumentácii o nástrojoch ANTLR a FLEX navrhni obsah kapitoly, ktorá opíše princíp fungovania tohto nástroja.“, ChatGPT, verzia GPT-5.3, OpenAI, citované 9.4.2026, chat.openai.com

Prílohy

Zoznam príloh

Príloha A | Používateľská príručka 21
Príloha B | Diagram tried jadra lexikálneho analyzátora 26
Príloha C | Diagram tried grafického rozhrania 27
Príloha D | Obsah DVD 28

Príloha A | Používateľská príručka

1 ÚVOD

Lexeur je desktopová aplikácia na lexikálnu analýzu zdrojového kódu. Jej úlohou je rozložiť vstupný text na tokeny – najmenšie významové jednotky jazyka (kľúčové slová, identifikátory, čísla, operátory a podobne) – podľa sady pravidiel definovaných regulárnymi výrazmi.

2 SPUSTENIE

2.1 Požiadavky

Tabuľka 1 – Požiadavky pre spustenie aplikácie

Požiadavka	Minimálna verzia	Poznámka
JDK	17	Napr. Oracle JDK
Apache Maven	3.6	

JavaFX, Gson a Junit nie je potrebné inštalovať ručne – Maven si ich stiahne sám.

2.2 Postup spustenia

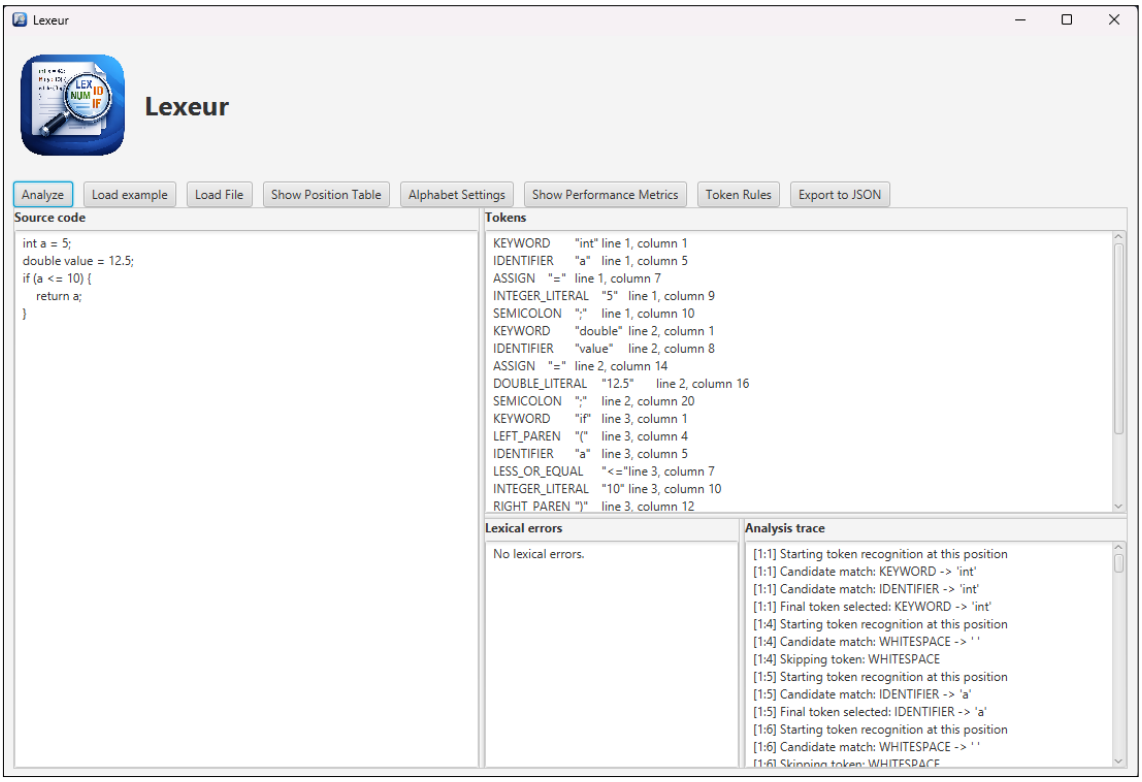
1. Otvorte terminál
2. Prejdite do priečinku projektu
3. Spustite aplikáciu príkazom

```
mvn javafx:run
```

4. Po úspešnej kompilácii sa otvorí hlavné okno aplikácie.

3 HLAVNÉ OKNO

Hlavné okno sa otvorí automaticky po spustení aplikácie. Je rozdelené na dve hlavné časti: ľavý panel so vstupným kódom a pravý panel s výsledkami analýzy.



Obrázok 3 – Hlavné okno aplikácie

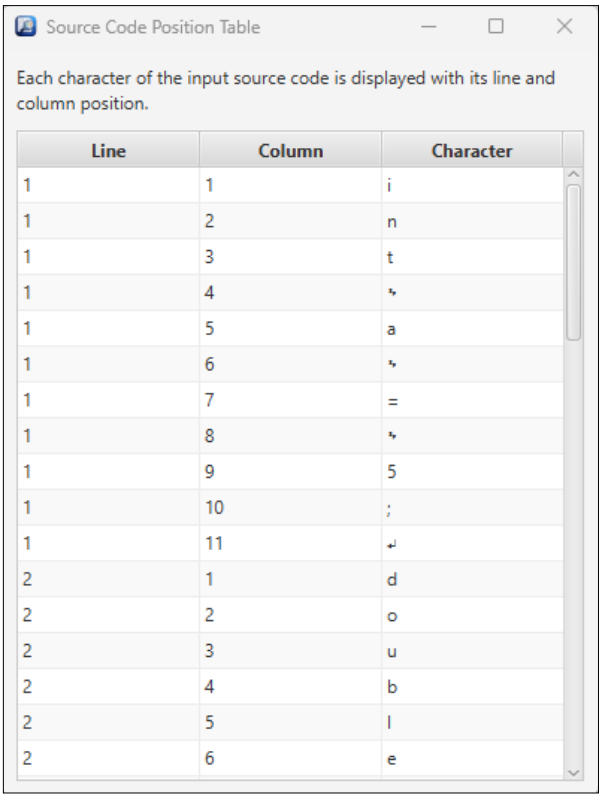
3.1 Tlačidlá

Tabuľka 2 – Funkcie jednotlivých tlačidiel

Tlačidlo	Funkcia
Analyze	Spustí lexikálnu analýzu nad aktuálnym obsahom textovej oblasti **Source code**. Výsledky sa zobrazia vo všetkých troch pravých paneloch.
Load example	Načíta pripravený ukážkový kód do oblasti **Source code**.
Load File	Otvorí prieskumník na výber súboru zo systému. Obsah vybraného súboru sa načíta do oblasti **Source code**.
Show Position Table	Otvorí okno Tabuľka pozícií (kapitola 4).
Alphabet Settings	Otvorí okno (kapitola 5).
Show Performance Metrics	Otvorí okno (kapitola 6). Tlačidlo funguje až po aspoň jednej úspešnej analýze.
Token Rules	Otvorí okno (kapitola 7).
Export to JSON	Uloží výsledky poslednej analýzy (tokeny aj chyby) do súboru vo formáte JSON.

4 TABUĽKA POZÍCIÍ

Okno **Source Code Position Table** zobrazuje vstupný zdrojový kód v tabuľkovej forme, kde každý znak má priradenú presnú pozíciu v texte – číslo riadku a číslo stĺpca.

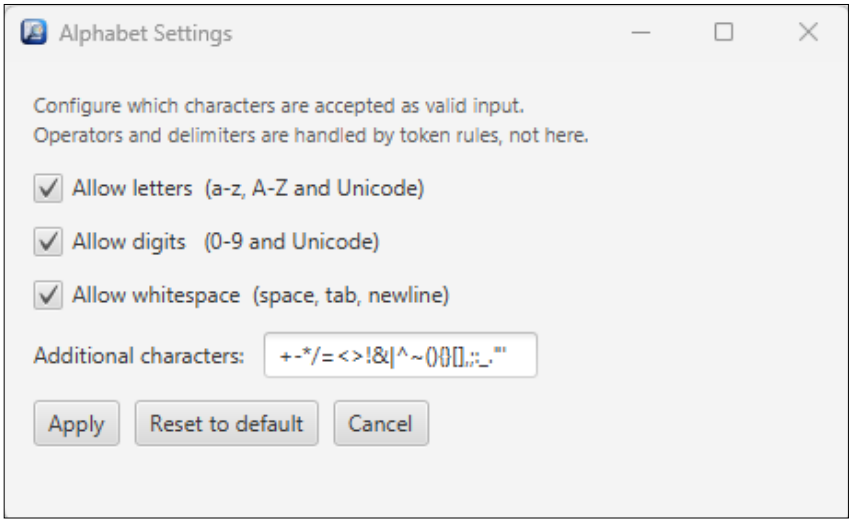


Line	Column	Character
1	1	i
1	2	n
1	3	t
1	4	␣
1	5	a
1	6	␣
1	7	=
1	8	␣
1	9	5
1	10	;
1	11	↵
2	1	d
2	2	o
2	3	u
2	4	b
2	5	l
2	6	e

Obrázok 4 – Tabuľka pozícií

5 NASTAVENIE ABECEDY

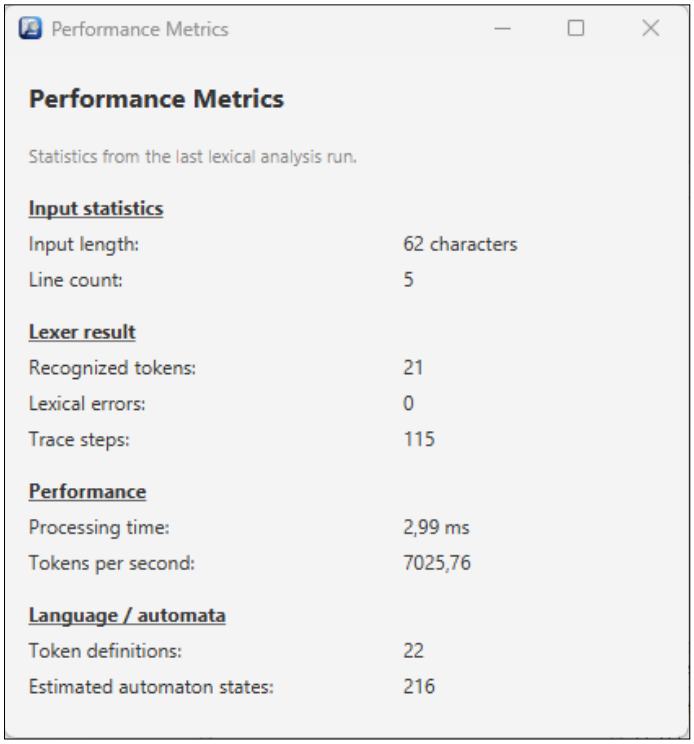
Okno **Alphabet Settings** definuje, ktoré znaky sú považované za platný vstup analyzátora. Znaky, ktoré nie sú súčasťou abecedy, spôsobia lexikálnu chybu.



Obrázok 5 – Nastavenie abecedy

6 VÝKONNOSTNÉ METRIKY

Okno **Performance Metrics** zobrazuje štatistiky a merania z posledného behu lexikálnej analýzy. Okno sa otvorí kliknutím na tlačidlo Show Performance Metrics v hlavnom okne.



Obrázok 6 – Výkonnostné metriky

Tabuľka 3 – Význam jednotlivých metrík

Metrika	Popis
Input length	Celkový počet znakov vstupného textu.
Line count	Počet riadkov vstupného textu.
Recognized tokens	Počet tokenov úspešne rozpoznaných analyzátorom (nezahŕňa preskočené tokeny).
Lexical errors	Počet znakov alebo reťazcov, ktoré sa nepodarilo priradiť k žiadnemu pravidlu.
Trace steps	Celkový počet krokov v stope analýzy – vyjadruje, koľko rozhodnutí analyzátor vykonal.
Processing time	Čas trvania analýzy v milisekundách.
Tokens per second	Priepustnosť analyzátora – počet rozpoznaných tokenov za sekundu.
Token definitions	Počet aktívnych tokenových pravidiel, ktoré analyzátor používal.
Estimated automaton states	Odhadovaný celkový počet stavov konečných automatov zodpovedajúcich všetkým pravidlám. Ide o teoretický odhad vypočítaný Thompsonovou konštrukciou.

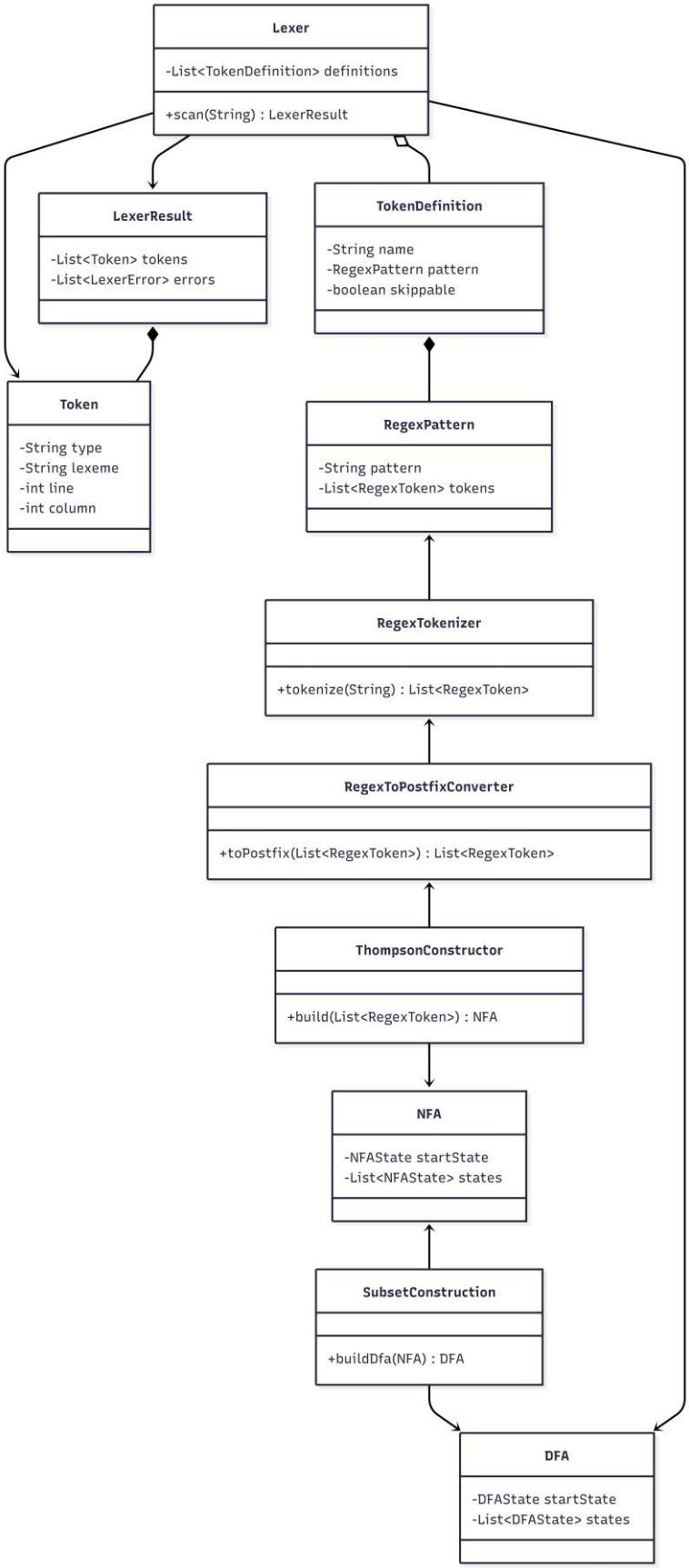
7 TOKENOVÉ PRAVIDLÁ

Okno **Token Rules Configuration** umožňuje definovať vlastné pravidlá rozpoznávania tokenov. Každé pravidlo pozostáva z názvu tokenu, regulárneho výrazu a doplnkových nastavení.

Tabuľka 4 – Význam jednotlivých polí v konfigurácii pravidiel tokenov

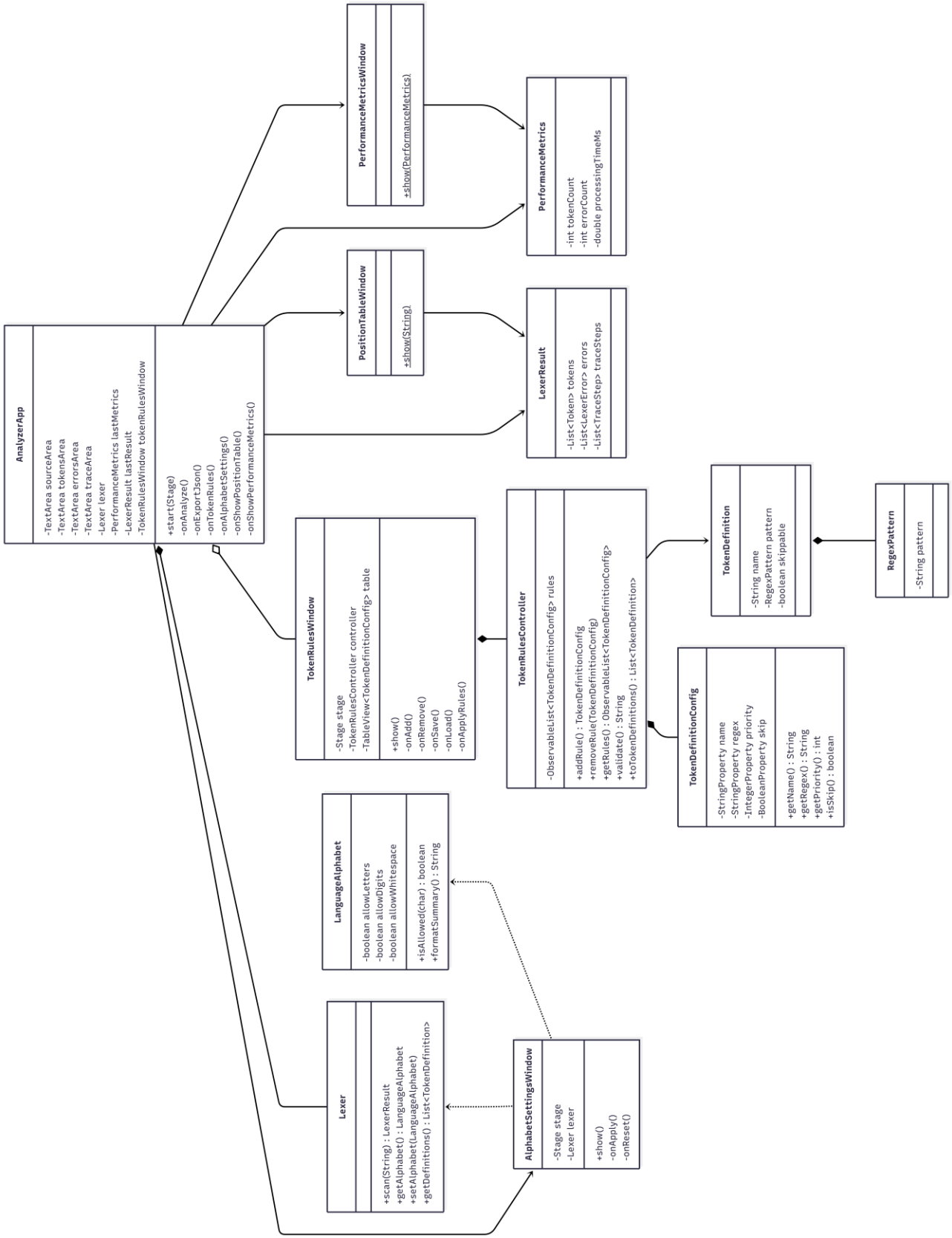
Stĺpec	Popis
Token Name	Názov typu tokenu, napríklad KEYWORD, NUMBER, IDENTIFIER.
Regex Pattern	Regulárny výraz, ktorý definuje tvar tokenu. Analyzátor sa pri každom znaku vstupného kódu pokúša tento vzor priradiť.
Priority	Celé číslo určujúce poradie vyhodnocovania pravidiel – nižšie číslo = vyššia priorita (pravidlo sa vyskúša skôr). Pri rovnakom zhode zvíťazí pravidlo s nižším číslom.
Skip	Ak je zaškrtnuté, token sa pri analýze rozpozná, ale nezahrnie sa do výsledného zoznamu tokenov.
Note	Nepovinná poznámka pre lepšiu orientáciu.

Príloha B | Diagram tried jadra lexikálneho analyzátora



Obrázok 7 – UML diagram tried jadra lexikálneho analyzátora

Príloha C | Diagram tried grafického rozhrania



Obrázok 8 – UML diagram tried používateľského rozhrania

Príloha D | Obsah DVD

Priložené DVD v sebe zahŕňa :

- Práca v elektronickej podobe
- Zdrojový kód projektu
- Konfiguračné .JSON súbory